2024 届研究生硕士学位论文

分类号：_____ 学校代码：10269

密　级：_____ 学　　号：51215901093



East China Normal University

硕士学位论文

MASTER'S DISSERTATION

论文题目：大规模集群下的 SSD 故障预测及其在 RAID 中的应用研究

院　　系： 计算机科学与技术学院

专　　业： 计算机技术

研 究 方 向： 存储可靠性

学位申请人： 梁宇炯

指 导 教 师： 石亮 教授

2024 年 5 月 5 日

中国知网 https://www.cnki.net

Dissertation for Master's Degree in 2024

University Code: 10269

Student ID: 51215901093

EAST CHINA NORMAL UNIVERSITY

SSD Failure Prediction under Large Scale Clustering and Its Application in RAID

Department:	School of Computer Science and Technology
Major:	Computer Technology
Research Direction:	Storage Reliability
Candidate:	Yujiong Liang
Supervisor:	Prof. Liang Shi

May, 2024

中国知网 https://www.cnki.net

梁宇炯 硕士学位论文答辩委员会成员名单

姓名	职称	单位	备注
殷树	副教授	上海科技大学	**主席**
魏同权	副教授	华东师范大学	
徐飞	教授	华东师范大学	

中国知网 https://www.cnki.net

摘　要

进入21世纪以来，互联网呈现出爆发式增长趋势，越来越多的应用服务出现，同时固态硬盘的技术不断成熟，容量增大，成本降低，使得数据中心使用固态硬盘的规模越来越多。随着规模增大，固态硬盘的故障逐渐变得不可忽视，常常会导致系统级故障，甚至造成数据中心宕机。如何通过预测方法预防故障带来的损失以及提升固态硬盘部署的可靠性成为了企业和学者研究的重点。

本文研究了华为数据中心超过20万块硬盘在5年内的故障特征以及日常数据。依据闪存类型、工作负载及其他S.M.A.R.T.属性的分类对比分析。提出了故障原因的差异可能反映在某些S.M.A.R.T.属性的数值分布差异上的新发现。进而提出了“状态”概念，即一组序列，这个序列中的各个值对应的是各个属性的分布区间。利用这种区分的方式减少数据集的混乱程度，同时潜在的区分一些故障原因。

此外，利用“状态”概念，本文还提出了Diff-State，一种自动数据预处理架构，以完善“状态”应用并提高故障预测的准确性。通过四个步骤即状态特征提取、属性值划分、”状态”合并以及最后的数据填充，能够将数据集切分为多个子数据集，并贴合不同的“状态”。再通过对不同的子数据集进行单独的建模，最终得到了不错的效果提升。在使用随机森林这类机器学习算法情况下，使用了Diff-State之后，当召回率达到0.91时，精确度可以保持在0.81，相比较没有使用Diff-State，精确度可以提升约0.4，召回率可以提升约0.35。同时，本文针对现有工作中对于样本选择的问题进行了优化，提出了带有恢复步长的回溯周期，很好的解决了健康样本不准确以及故障样本不及时的两个问题。

最后，在数据中心常见的RAID应用架构下，本文提出了预恢复结合故障预测技术来代替传统RAID恢复，以减少RAID系统的恢复数据时的降级时间。目前的预测方法强调精确性而忽视召回性，这限制了预测手段对故障设备的识别能力。为解决这一问题，本文提出了一种两阶段备份机制，以提高预测的召回率，同时避免

中国知网　https://www.cnki.net

替换被误判为健康的存储设备。最终的评估结果表明，两阶段备份机制可以显著减少数据中心的故障时间。通过预恢复的方式，降级性能仅仅只下降了约 150MB/s，相比较之下，传统 RAID 恢复机制则会有近 4000MB/s 的性能损失。同时，从集群角度来看，在总体 70 万台设备和 1.6 万台故障设备的情况下，预恢复这种推崇搞召回率的方式，在仅增加了 481 台无效备份数量的情况下，可以减少总体 250000 分钟的降级时间。这给上层应用的运行稳定性带来了极强的保护。

关键词：数据中心、固态硬盘、S.M.A.R.T.、故障预测、RAID

中国知网 https://www.cnki.net

ABSTRACT

Since entering the 21st century, the Internet has shown an explosive growth trend, with more and more application services appearing. At the same time, the technology of SSDs has been maturing, with increased capacity and lower cost, making data centers use SSDs on a larger and larger scale. As the scale increases, SSD failures gradually become unnoticeable, often leading to system-level failures and even causing data center downtime. How to prevent losses from failures through predictive methods and improve the reliability of SSD deployment have become the focus of research by enterprises and scholars.

This article examines the failure characteristics of over 200,000 hard drives in Huawei's data centers over a 5-year period, as well as daily data. A comparative analysis is conducted based on the classification of flash types, workloads, and other S.M.A.R.T. attributes. Suggests that differences in failure causes may be reflected in differences in the distribution of values for certain S.M.A.R.T. attributes. The concept of "state" is then proposed, i.e., a set of sequences in which the individual values correspond to the distribution intervals of each attribute. This differentiation is utilized to reduce the level of confusion in the dataset, while potentially distinguishing some of the causes of failure.

In addition, utilizing the concept of "state", this article also proposes Diff-State, an automated data preprocessing architecture to refine the application of "state" and improve the accuracy of fault prediction. Through four steps, i.e., state feature extraction, attribute value partitioning, "state" merging, and finally data populating, the dataset can be sliced into multiple sub-datasets and labeled with different "states". By modeling the different sub-datasets individually, the results were improved. In the case of machine learning algorithms such as Random Forests, the precision can be maintained at 0.81 when the recall reaches 0.91 with the use of Diff-State, and the precision can be improved by about 0.4 and the recall can be improved by about 0.35 compared with that without the use of Diff-

中国知网 https://www.cnki.net

State.Meanwhile, this article optimizes the problem of sample selection in the existing work, and proposes a recovery step with a recovery step. A backtracking cycle with recovery step is proposed, which well solves the two problems of inaccurate healthy samples as well as untimely faulty samples.

Finally, under the common RAID application architecture in data centers, this article proposes pre-recovery combined with failure prediction techniques to replace traditional RAID recovery in order to reduce the degradation time when recovering data from RAID systems. Current prediction methods emphasize precision and ignore recall, which limits the ability of prediction means to identify faulty devices. To address this issue, this article proposes a two-stage backup mechanism to improve the recall of predictions while avoiding the replacement of storage devices that are misidentified as healthy. The final evaluation results show that the two-phase backup mechanism can significantly reduce data center downtime. With pre-recovery, degradation performance drops by only about 150MB/s, compared to a performance loss of nearly 4000MB/s with traditional RAID recovery mechanisms. Meanwhile, from a cluster perspective, with 700,000 devices overall and 16,000 failed devices, pre-recovery, which pushes the recall rate, reduces the overall degradation time by 250,000 minutes while increasing the number of invalid backups by only 481. This brings extreme protection to the operational stability of upper tier applications.

Keywords: *Data Center; Solid State Drive; Self-Monitoring Analysis and Report Technology; Failure Prediction; RAID*

中国知网 https://www.cnki.net

目 录

第一章 绪论.....1
1.1 研究背景及意义.....1
1.1.1 研究背景.....1
1.1.2 研究意义.....2
1.2 国内外研究现状.....3
1.2.1 大规模集群下的故障 SSD 运维数据分析的研究.....3
1.2.2 大规模集群下 SSD 故障预测的研究.....5
1.3 研究目标和内容.....7
1.4 论文组织结构.....9
第二章 相关背景介绍.....11
2.1 SSD 设备基本架构及错误原理.....11
2.1.1 SSD 基本架构.....11
2.1.2 常见的 SSD 错误产生原因.....13
2.2 基于机器学习的故障预测技术.....16
2.2.1 故障预测技术的基本流程.....16
2.2.2 预测模型的挑战.....18
2.3 基于奇偶校验的 RAID 冗余技术.....19
2.3.1 RAID 机制的基本原理.....19
2.3.2 常见 RAID 架构.....21
2.4 本章小结.....24
第三章 数据分析：数据中心故障 SSD 的运维数据分析.....25
3.1 数据集来源及基本信息.....25
3.1.1 SSD 类别区分.....25
3.1.2 S.M.A.R.T. 内容介绍.....26
3.1.3 基于时间的故障分布情况.....28
3.2 基于 S.M.A.R.T. 信息的故障差异分析.....29
3.2.1 MLC & TLC 差异.....29
3.2.2 早期死亡率.....31
3.2.3 一些其他差异及总结.....33
3.3 基于 S.M.A.R.T. 属性差异的 SSD“状态”定义.....33
3.4 本章小结.....35
第四章 性能优化：差异化的机器学习 SSD 故障预测研究.....37
4.1 研究动机：数据中心 SSD 故障预测性能不佳.....37
4.2 设计：自动化数据预处理架构 Diff-State.....38

中国知网 https://www.cnki.net

4.2.1 状态属性选取.....39
4.2.2 属性值区分.....40
4.2.3 “状态”合并.....42
4.2.4 数据填充.....43
4.3 实验与分析.....43
4.3.1 实验设置.....44
4.3.2 Diff-State 的故障预测性能评估.....45
4.3.3 Diff-State“状态”区分有效性的验证.....47
4.3.4 Diff-State 模型学习时间评估.....48
4.4 本章小结.....49
第五章 采样优化：预测模型的回溯周期优化.....51
5.1 研究动机：训练样本选择不准确.....51
5.1.1 常见样本选择方式.....51
5.1.2 样本选择问题分析.....52
5.2 设计：带有恢复步长的样本选择优化.....54
5.2.1 故障样本优化情况.....54
5.2.2 健康样本优化情况.....55
5.2.3 恢复步长对预测性能的影响分析.....55
5.3 实验与测试.....57
5.3.1 实验配置.....57
5.3.2 恢复步长对预测性能的测试评估.....57
第六章 部署优化：基于预测备份的 RAID 快速恢复技术.....59
6.1 研究动机：预测部署受限无法发挥最大性能.....59
6.1.1 动机一：预测部署受限.....59
6.1.2 动机二：数据中心常见场景 RAID 数据恢复速度慢.....59
6.2 设计：两阶段的故障预恢复架构 Precovery.....60
6.2.1 模型调整.....61
6.2.2 预测失效设备的镜像生成.....62
6.2.3 异常事件处理.....63
6.3 实验与测试.....68
6.3.1 单 RAID 系统测试.....68
6.3.2 集群规模评估.....72
6.4 本章小结.....75
第七章 总结与展望.....77
7.1 工作总结.....77
7.2 未来展望.....78
参考文献.....79

中国知网 https://www.cnki.net

图目录

图 1.1 本文研究内容框图 …… 8

图 2.1 常见 SSD 架构 …… 11
图 2.2 基于机器学习的 SSD 故障预测常见流程 …… 17
图 2.3 PRE 和 RECALL 折衷示意图 …… 19
图 2.4 RAID4 及 RAID5 架构 …… 20
图 2.5 传统 RAID0 及 RAID1 架构 …… 21
图 2.6 传统 RAID6 架构 …… 22
图 2.7 传统 RAID1+0 和 RAID0+1 架构 …… 23

图 3.1 各类型 SSD 的故障在时间跨度上的分布 …… 28
图 3.2 不同 SSD 类型的出场坏块数分布 …… 29
图 3.3 MLC 与 TLC 在三种工作负载属性上的累积分布差异 …… 31
图 3.4 Fbb 数值在设备不同年龄阶段的分布。条形图的水平线为中位值，绿色三角形为平均值 …… 32

图 4.1 Diff-State 的整体架构 …… 38
图 4.2 各类方法使用 Diff-State 或者不使用时的预测性能比较，折线代表各种方法在 FPR 为 0.1% 时的 TPR 值 …… 46
图 4.3 使用算法 1 及不使用时的预测性能比较, 折线代表各种方法的 TPR 值 · 48

图 5.1 三种不同的回溯周期 …… 52
图 5.2 故障发生当日附近的负载状况。纵坐标以百分比为单位，横坐标为时间步长，即故障发生前的天数。 …… 53
图 5.3 标记故障样本和健康样本的情况 …… 56
图 5.4 不同回溯周期大小和恢复步长下的预测性能, 曲线代表性能趋势 …… 58

中国知网 https://www.cnki.net

图 6.1 两阶段的故障预恢复架构示意图 …… 61
图 6.2 镜像设备的生命周期 …… 63
图 6.3 流程中可能存在的异常事件 …… 64
图 6.4 多正样本设备解决情况 …… 65
图 6.5 观察期和后续过程 …… 67
图 6.6 相对于正常吞吐量的降级吞吐量（越高越好） …… 69
图 6.7 4KB 随机读取吞吐量的降级。对于 x 轴上的标签，*d* 指队列深度，*t* 指线程数，后面的数字分别是它们的值 …… 70
图 6.8 设备故障和镜像生成对 4KB 随机读取吞吐量的瞬时影响 …… 70
图 6.9 不同模型状态下的精确度和召回率趋势。标出了三个标志性的点，旁边列出的是其精确度和召回率 …… 72
图 6.10 集群数学建模下的收益和开销对比 …… 74
图 6.11 集群数学建模下的收益和开销对比趋势，图中标记了一个推荐配置点 …… 74

中国知网 https://www.cnki.net

表目录

表 3.1 SSD 分类信息 ····· 26
表 3.2 本文关注的 S.M.A.R.T. 信息列表 ····· 27
表 3.3 “状态” 定义 ····· 34

表 4.1 各类预测模型在不同数据集下的性能对比 ····· 37
表 4.2 故障相关度最高的 7 个属性 ····· 40
表 4.3 新旧设备最相关的 6 个 S.M.A.R.T. 属性 ····· 47
表 4.4 算法 1 后得到的 7 个最相关属性的 P_value ····· 48
表 4.5 模型训练时间 ····· 49

表 6.1 RAID5 降级的性能表现 ····· 60
表 6.2 降级时间情况，前台负载为 4kb 随机读写负载 ····· 71

中国知网 https://www.cnki.net

第一章 绪 论

1.1 研究背景及意义

进入 21 世纪以来，互联网呈现出爆发式增长趋势，越来越多的应用服务出现，成为人们生活中不可缺少的一部分。尤其是人工智能、直播、云计算等新技术的快速发展，给数据存储带来了极大的挑战。为了满足大量数据存储的需求，IDC（Intel Data Center，互联网数据中心）应运而生。据中国信通院统计分析，在中国，2022 年的 IDC 机架规模就已经超过 670 万架，同时还保持着高速增长趋势，每年的符合增长率高于 30%。在如此大的规模下，数据的可靠性成为了极大的挑战。如何在 IDC 中的数据不丢失的同时保证服务质量成为一个难题。

1.1.1 研究背景

为了应对大量数据存储的需求，早期 IDC 通常会使用机械硬盘（Hard Dsik Drive，简称 HDD）来写入数据，由于 HDD 是通过磁性记录，利用磁场的形式将数据存储，因此也常常被称为磁盘。HDD 的单位存储容量价格低，同时离电之后保存的时间也很长，这让其在 IDC 场景下有着一定的优势。然而，同样的，HDD 也面临着其他的挑战：（1）耐用性较差，很容易收到撞击和震动的影响。（2）由于其物理结构中存在的电动马达，运行时的整体功耗较高。（3）旋转磁片来读取数据的原理，意味着其糟糕的随机读写能力。（4）机械结构所带来的可靠性问题，年故障率较高，可以达到 2-9%[1, 2]。尤其是可靠性挑战，由于其年故障率较高，IDC 往往需要冗余手段来保证数据的完整不丢失，比如磁盘阵列（Redundant Arrays of Independent Disks，简称 RAID）或纠删码技术。然而，冗余手段虽然能够保护数据，但是在恢复丢失数据的过程中会出现性能降级的问题，影响上层应用运行。

目前，随着基于闪存的固态盘（Flash-base Soild State Drive, 简称 SSD）技术越来越成熟，单位存储容量价格不断下降，数据中心越来越偏向于使用 SSD 进行数据存储来解决上述挑战。相比较于 HDD，尽管 SSD 的可靠性有一定的提升，年故障率可以降低到 1-2.5%[3]，但是在设备规模不断增大的前提下，这依旧是一个难

中国知网 https://www.cnki.net

以忽视的数字。为此，基于预测的故障处理手段开始兴起。这种手段寄希望于在SSD 还未真实发生故障前将其识别出来，提前迁移数据来保证数据的完整性和上层服务的连续性。然而，由于全闪阵列（All Flash Array，简称 AFA）推广时间并不久，同时 SSD 的物理原理与 HDD 完全不同，这导致过去几十年针对于 HDD 的故障分析工作无法应用于 SSD。因此，基于预测的手段往往效果不佳，尽管各大企业比如谷歌、微软、阿里巴巴等企业都有相关研究，但是并没有形成一个较为完善的运作体系。目前，在数据中心环境下 SSD 故障预测以及预测应用上依旧存在较大的缺口。

1.1.2 研究意义

本文主要的研究意义是在真实数据中心场景下，利用真实获得的运维数据，对实际发生的 SSD 故障特性进行分析研究。同时依据特性分析所得，提升 SSD 故障预测性能，并且完善故障预测部署机制，让其更适合在实际场景中发挥作用。最后，在 RAID5 这种数据中心常见场景下，提出预恢复机制，将预测手段和备份手段相结合，充分利用预测技术，极大改善了 RAID5 本身数据恢复速度慢的问题，这在降低数据中心 SSD 运维成本的同时，保证了数据中心的数据可靠性以及服务的稳定性。论文主要研究意义如下：

（1）对真实数据中心场景下的超过 20 万个 SSD 数据，横跨近 5 年的运维数据进行了数理分析研究，同时提出了基于机器学习算法的 SSD 故障预测技术并加以改进提升预测性能。

（2）基于对运维数据的分析研究，提出了 SSD “状态”的概念，通过“状态”的概念，将潜在的 SSD 故障原因进行区分，为 SSD 故障分析提供了新的思路。

（3）提出了 Diff-State，这是一个自动化的数据预处理流程，以完善“状态”应用并提高故障预测的准确性。通过四个步骤即状态特征提取、属性值划分、”状态”合并以及最后的数据填充，能够将数据集切分为多个子数据集，并贴合不同的“状态”。最终对不同的子数据集进行差异化训练，有效提升了 SSD 故障预测性能。

（4）提出了 Precovery-raid，将预测手段实际用于 RAID，通过备份预恢复的方式，能够很好的避免数据恢复过程中的性能降级问题，在保护数据的同时保证系

中国知网 https://www.cnki.net

统运行性能的稳定。

1.2 国内外研究现状

1.2.1 大规模集群下的故障 SSD 运维数据分析的研究

早期分析存储系统故障的工作集中在 HDD。然而由于实现原理不同，先前工作得到的结论并不能完全应用于基于闪存的 SSD。目前，一些相关研究都集中在探求 SSD 故障发生规律上。

Meza[4] 等人首次基于闪存的 SSD 的可靠性进行了大规模的研究，从内外两个角度探究各自因素与 SSD 故障之间的关系。并得到一下结论：（1）SSD 失效率并不随闪存芯片磨损单调增加。（2）读干扰错误的影响在该领域并不普遍。（3）SSD 物理地址空间 (例如, 非连续数据) 上的稀疏逻辑数据布局，可以极大地影响 SSD 的失效率。（4）较高的温度导致较高的失效率，但抑制 SSD 操作的技术似乎可以极大地降低较高温度的负可靠性影响。（5）操作系统写入到基于闪存的 SSD 的数据并不总是被准确的展示。这些结论作为首次针对于数据中心环境下 SSD 故障的研究分析，展示了这种场景下故障表现的独特性。

Schroeder[3] 等人进行了一项大规模的实地研究，覆盖了数百万个设备使用日，在谷歌数据中心生产使用 6 年以上，包含 10 种不同的驱动模型，以及三种不同的闪存技术。基于这些数据，研究了广泛的可靠性特征，并得出了一些意想不到的结论。（1）原始误比特率 (RBER) 的增长速度比通常假设的指数速度要慢得多，更重要的是，它们不能预测不可纠正的错误或其他错误模式。（2）广泛使用的度量指标 UBER (不可纠正的误码率) 并不是一个有意义的度量指标，因为没有看到读数与不可校正错误数之间的相关性。（3）没有证据表明，在典型的设备寿命内，更高端的设备更可靠。（4）与传统硬盘驱动器相比，闪存盘在现场的替换率显著较低，但不可修正的出错率较高。

Narayanan [5] 等人除了关注于生产数据中心中 SSD 的故障表征外，首次关注到了 SSD 故障的时间和原因。他们分析来自超过五十万个 SSD 的广泛数据，展示了广泛的 SSD 故障特征，这些 SSD 跨越了多个数据中心的多代，这些数据中心在近 3 年的时间里接受了各种各样广泛的工作负载。基于这样的真实数据，对故障

中国知网 https://www.cnki.net

的各种设计、配置和操作因素及其症状做了较为全面的分析。

Alter[6] 等人同样利用谷歌数据中心的数据集描述了导致故障的工作负载条件，并说明了它们的根本原因不同于通常的期望，但仍然难以识别。特别地，他们还研究了修复过程中导致人工干预的故障事件，这是一个全新的研究角度。

Xu [7] 等人则是基于运维角度来寻找面对数据中心场景 SSD 故障的应对手段。他们研究了阿里云中部署的基于 SSD 的存储系统的可靠性，这些系统覆盖了近五十万个 SSD，并在具有代表性的云服务下使用了三年多。通过日志分析、现场研究和验证实验，从分布、症状和相关性方面确定了 SSD 故障的特征。最重要的是总结了一些重要的经验教训和一套有效的方法来解决所观察到的问题。这对于数据中心的运维、故障处理有着极大的帮助。

Maneas[8] 等人则基于另外一套数据集做了对应的研究，该数据集来自于存储供应商 NetApp 的 140 万个 SSD，包括 3 个不同的制造商，18 个不同的型号，12 个不同的容量，以及四种不同的闪存技术。借此研究了许多以前工作中没有研究的因素，包括固件版本的影响，NAND 介质的可靠性，以及 RAID 系统中驱动之间的相关性。

Han and Lee[9] 等人认为 SSD 的相关故障非常重要，这里提到的相关故障是指时间上以及空间上十分接近的故障。他们利用阿里巴巴数据中心实际获得的运维数据，对于相关故障进行研究，总计可达 11 个 SSD 设备类型，包含超过百万个 SSD 的总数量。最终的研究结果表明，空间上接近的相关故障确实十分常见，也就是说会有相当一部分的故障其实发生在同一个节点上或者是同一个机架上。这是第一次对故障的相关性做了比较详细的研究。

Li [10] 等人首次研究了系统级的基于 NAND 闪存的 3D 三层单元（3D triple-level cell，简称 3D TLC）SSD。这种技术的 SSD 存储密度高、每比特成本低，正逐渐成为大规模存储系统中占主导地位的存储介质，但是由于其巨大的数量，很大程度上也增加了故障概率，导致了很多不可逆转的数据丢失和服务不可用的情况。他们以基于现场自我监控、分析和报告技术（Self-Monitoring, Analysis and Reporting Technology，简称 S.M.A.R.T.）数据来表征可靠性和亚健康状态。通过探索真实世界的数据集，并在预定类别中为每个选定的属性得出一些发现，有助于后续的特

中国知网 https://www.cnki.net

征选择以及预测模型的可解释性。

1.2.2 大规模集群下 SSD 故障预测的研究

除了在大规模真实场景下研究 SSD 的故障特性。也有不少工作在尝试故障预测上做出了努力。目前的故障预测基本都会结合机器学习算法进行。这是因为，对于 SSD 故障预测来说，与其相关的因素太多，包含负载、介质、环境、电路等等，过多的因素对最后的判断造成了影响。普通的阈值判断无法有效的区分健康设备和故障设备。而机器学习能够比较好的面对这一类复杂的规约问题，通过对大量数据的不断拟合，进而寻找到潜在的规律。

通常来说，按照学习类型来分，机器学习可以分为监督学习和无监督学习。在 SSD 故障领域，监督学习就是提前给数据打好标签，即为机器学习模型指出故障设备是哪些，让模型寻找故障设备和健康设备之间的差异。而无监督学习则相反，不提供标签，让模型去寻找和大部分数据差异较大的离群点，往往这些离群点就会是故障的设备样本。目前这两个方面都有相关的研究工作。

有监督学习

在监督学习方面，Xu [11] 等人关注于数据类型，对于 S.M.A.R.T. 来说，由于其覆盖方面较多，包含工作负载、介质情况、环境因素、电路问题等各个方面，因此能够提供的数据属性特别多。但是这些数据属性并不都对故障产生影响。将这些不相关的因素投入到模型中，不仅会增加模型的计算时间，甚至可能影响最后的预测性能。因此他们专注于属性选择，使用肯德尔相关系数距离来综合目前现有的多种特征选择技术，肯德尔相关系数往往用来测量两个随机变量的相关性。通过这种方式增强鲁棒性，以达到在精简模型的同时提升性能。

Zhang [12] 等人则专注于寻找更多的数据组织形式。他们提出一种多视角、多任务随机森林（MVTRF）方案，从 SSD 长期和短期监控数据中提取了多视图特征，同时创新型的采用了多任务学习，让模型同时预测故障类型和故障何时发生。除此之外，基于 MVTRF 的一些关键决策，团队分析了具体的故障发生原因，这对于进一步验证和处理很有帮助。在真实的数据中心的大规模数据验证中，MVTRF 能

中国知网 https://www.cnki.net

够得到更高的失效预测精度，与现有方案相比，平均精度提高了 46.1%，召回率提高了 57.4%。

Han [13] 等人考虑到了数据收集的不准确性和不完整性，即收集到的数据可能存在的三种情况：标签不准确、数据样本缺失和有些故障类型无法被识别。在现实世界的生产环境中，这种数据的不完美情况很容易影响预测的准确性。因此，其提出了 RODMAN，一种鲁棒的数据预处理流程，在数据输入到模型之前对其进行完善，解决前述的三个问题。通过对阿里云数据中心的三百多万块磁盘进行的大规模跟踪研究，设计了具有三种数据预处理技术的 RODMAN，即故障类型过滤、基于样条线的数据填充和故障前自动回溯，使得最终的模型评估相对得到了提升。

除了解决数据不完美的情况，Han [14] 等人也尝试了使用不同类型的机器学习算法。通常来说，先前介绍的方式都可以称之为离线模型。这种方式的特点就是使用大量的数据集，通过长期保存下来的数据，来找寻其中可能存在的故障特征。因为数据量巨大，因此这类方式的模型通常更新并不频繁。这种方式很适合日志数据量大、负载稳定的情况。与之相对的，当工作负载经常变化时，对设备的压力也在不断变化，这可能导致故障的表现也随之发生变化。此时，对于离线学习就是一个巨大的挑战，因为更早期的规律会不可避免的干扰当下的模型归约。因此，Han 等人尝试使用自适应随机森林（Adaptive random forests，简称 ARF）、类 Boosting 在线集成（Boosting-like online ensemble，简称 BOLE）、具有漂移检测的快速增量模型树（Fast incremental model trees with drift detection，简称 FIMT-DD）等增量机器学习方式来实现在线学习。他们提出了 STREAMDFP，这是一种用于磁盘故障预测的通用流挖掘框架，具有概念漂移适应功能。STREAMDFP，采用了三种关键技术，即（i）在线标记、（ii）概念漂移感知训练和（iii）一般预测，其主要目标是使 STREAMDFP 支持各种增量机器学习算法。在最后的评估表明中，在各种设置下，STREAMDFP 与没有自适应的情况相比，显着提高了预测精度，并实现了相当高的流处理性能。

中国知网 https://www.cnki.net

无监督学习

在无监督学习方面，Chakraborttii[15] 提出使用提出了使用一分类模型来提升模型的准确性、适用性和可解释性。他们分析了六年来在 Google 数据中心运行实时应用程序的超过 30,000 个 SSD 的遥测数据，使用机器学习技术来预测和解释 SSD 故障。他们考虑了可能出现以前未见过的 SSD 类型这种情况，对于有监督学习来说，面对没有出现过的故障，模型没有办法识别，因为有监督学习需要通过先前大量的被打上标签的故障样本来学习故障的规律，在没有先前经验的情况下，模型在这种情况下效果就会变得很差。但是无监督可以比较好的解决这类问题，因为无监督本质只需要当下的数据情况，因此 Chakraborttii 建议使用一类隔离森林和基于自动编码器的异常检测技术来高精度预测以前未见过的 SSD 故障类型。最后的评估表明，忽略少数类别进行训练可以将性能提高高达 9.5%，如果需要适应动态环境，则可以提高高达 13%。此外，还创新型的利用一类自动编码器来实现模型的可解释性。特别是，基于自动编码器的方法可以推理导致 SSD 故障的原因。

1.3 研究目标和内容

在当前 IDC 中,越来越多的使用全闪阵列来代替之前的 HDD 阵列。但是由于故障导致的数据丢失进而影响整体系统性能甚至造成宕机的挑战十分严峻。图1.1展示了本文研究内容的组织架构，本文针对 IDC 场景下的 SSD 故障，利用真实场景下的数据，研究其故障特性，揭示一些和实验室场景下不同的规律，并提出新的故障预测思路。同时探索在真实应用场景下，如何发挥故障预测的最佳作用。

论文将依靠华为真实数据中心下的运维数据，主要研究内容如下：（1）论文研究了真实数据中心场景下的 SSD 故障信息，从多个角度分析故障和哪些因素相关，并且找寻其与实验室场景下的差异。（2）在此之上，论文提出了 “状态” 概念。通过对与故障相关的重要属性的值区间进行划分，进而潜在性的区分不同故障原因。并通过这种方式来优化传统的故障预测手段。（3）为了合理使用 “状态” 区分，论文提出了一种自动化数据预处理流程 Diff-State，通过特征筛选、特征值区分、“状态” 设置、回溯周期设置四个步骤改进了传统 SSD 故障预测流程。（4）探索故障预

中国知网 https://www.cnki.net

测在真实场景下的部署应用，提出了 Precovery-raid，提出了备份的概念，优化了 RAID 在恢复数据时的降级情况，同时也进一步优化了预测算法实际使用时的性能选择。

本文的主要创新点如下：（1）依据华为 IDC 真实场景下的真实数据，提出了”状态“的概念。通过”状态“寻找潜在的难以区分的故障原因。(2) 提出了 Diff-State，一种自动化的数据预处理流程，实现了”状态“的具体划分，同时优化了传统 SSD 故障预测流程。提升了故障预测的性能。（3）提出了 Precovery-raid，优化了传统 RAID 架构，使之与故障预测相结合。减少了集群下 RAID 由于数据恢复带来的降级影响。同时优化了故障预测的实际部署规则。

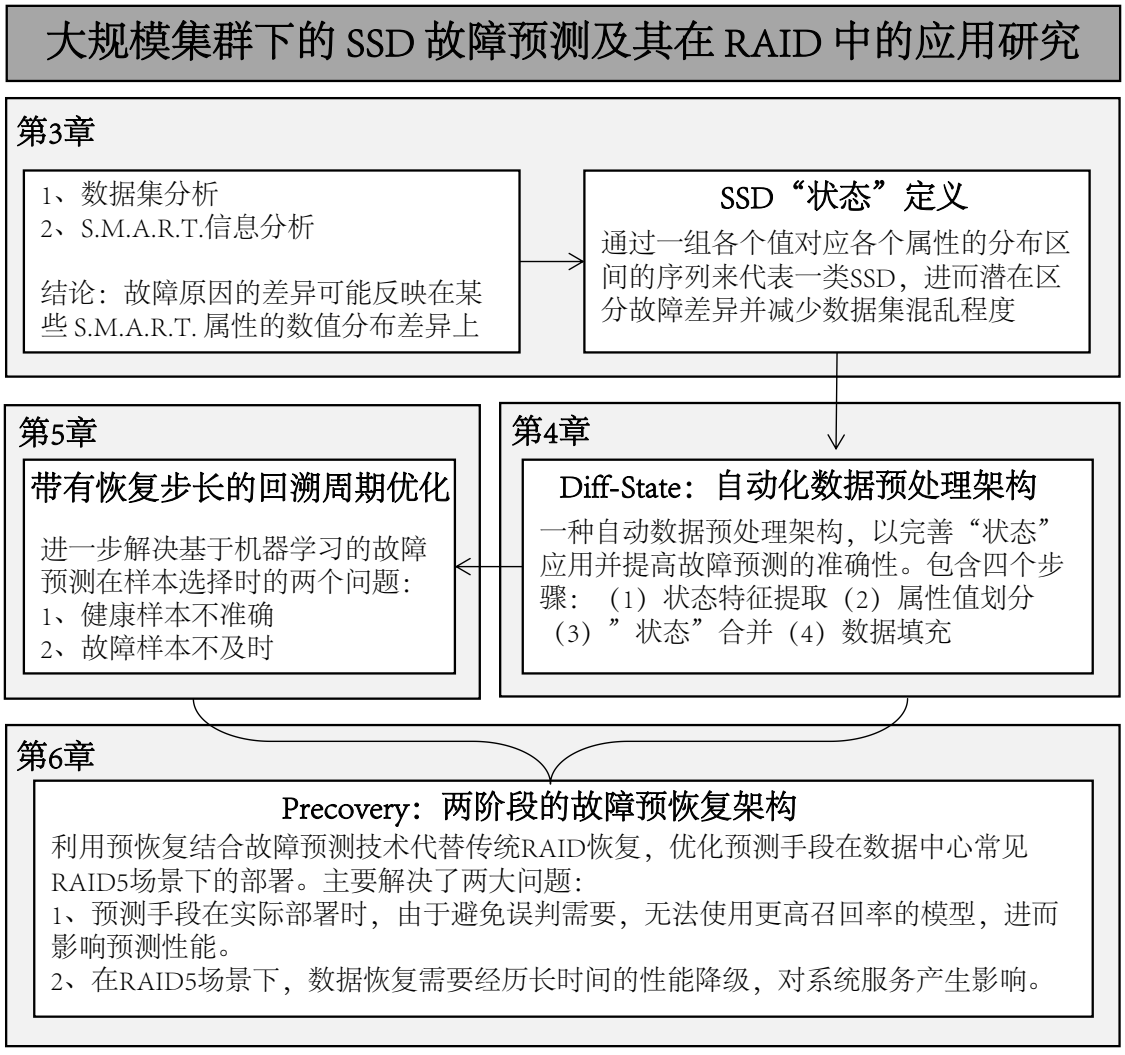


图 1.1 本文研究内容框图

中国知网 https://www.cnki.net

1.4 论文组织结构

本文主要分为 7 个部分，具体内容如下：

第一部分是绪论部分，给出了数据中心 SSD 故障分析及预测的研究背景、意义及进展工作，同时，给出了论文的研究内容和创新点。

第二部分给出了本研究使用的技术背景及需要解决问题的已有解决方案。首先主要介绍了 SSD 的一些物理原理以及架构情况，进而引出 SSD 本身如何出现错误。其次，介绍了针对数据中心 SSD 故障这个问题目前业界正在研究的解决方案，即利用机器学习方法来进行预测，进而对数据进行提前保护，保障上层应用服务的稳定性。本章节中详细介绍了基于机器学习的故障预测目前常见的流程以及存在的挑战。最后，针对于数据中心常见的应用场景即 RAID，介绍了其实现原理以及存在的缺陷。

第三部分首先研究了华为真实数据中心中收集到的 S.M.A.R.T. 数据集，介绍了数据集的基本情况，并且通过 NAND 类型、容量、数量等合理的将接近 200000 块 SSD 分成了六类。同时，详细介绍了收集到的数据集中涉及的属性类型，依据针对性分成了四类，即固有属性、错误相关属性、工作负载相关属性和持续时间及磨损属性。其次重点对收集到的数据集进行了分析研究，通过对于闪存类型、工作负载及其他 S.M.A.R.T. 属性的分类对比分析，可以发现故障原因的差异可能反映在某些 S.M.A.R.T. 属性的数值分布差异上。进而提出了“状态”概念，同时详细介绍了状态的生成过程。状态从本质上来讲就是一组序列，这个序列中的各个值对应的是各个属性的分布区间。状态区分的方式不仅可以减少数据集的混乱程度，同样也能潜在的区分一些故障原因。这为后续的数据预处理研究提供基础。

第四部分提出了 Diff-State，一种自动数据预处理架构，以完善“状态”应用并提高故障预测的准确性。通过四个步骤即状态特征提取、属性值划分、”状态”合并以及最后的数据填充，能够将数据集切分为多个子数据集，并贴合不同的“状态”。再通过对不同的子数据集进行单独的建模，最终得到了不错的效果提升。在随机森林算法中，使用了 Diff-State 之后，当 RECALL 达到 0.91 时，PRE 可以保持在 0.81，相比较没有使用 Diff-State，PRE 可以提升约 0.4，RECALL 可以提升约

中国知网 https://www.cnki.net

0.35。最后，用实验证明了 Diff-State 并不会增加整体运行时间，相反，由于分割之后，数据集规模被分割，有利于模型的搭建，整体运行时间反而缩减。

第五部分针对现有工作中对于样本选择的问题进行了优化，提出了带有恢复步长的回溯周期，很好的解决了健康样本不准确以及故障样本不及时的两个问题。并通过实验展示了在不同的恢复步长以及回溯周期下，模型性能的变化情况。尽管设置恢复步长之后会带来一定的性能损失，但是这使得最终的性能更加接近真实情况。

第六部分提出了预恢复结合故障预测技术来代替传统 RAID 恢复，以减少 RAID 系统的降级时间。目前的预测方法强调精确性而忽视召回性，这限制了故障前恢复对总降级时间的减少。为解决这一问题，本文提出了一种两阶段备份机制，以提高预测的召回率，同时避免替换被误判为健康的存储设备。本章还讨论了异常情况，以证明所提方法的鲁棒性。最终的评估结果表明，两阶段备份机制可以显著减少数据中心的故障时间。通过预恢复的方式，降级性能仅仅只下降了约 150MB/s，相比较之下，传统 RAID 恢复机制则会有近 4000MB/s 的性能损失。同时，从集群角度来看，预恢复这种推崇搞召回率的方式，在仅增加了 481 台无效备份数量的情况下，可以多获得 150% 的收益提升。

第七部分对论文进行总结与展望，给出了论文的结论，同时对研究缺陷和下一步将要进行的工作进行阐述。

中国知网 https://www.cnki.net

第二章　相关背景介绍

本章节详细介绍了与本文相关的研究技术及原理，以便读者深入理解本文所研究的工作。首先，第 2.1 小节主要介绍了 SSD 设备的基本架构及其发生错误的一些情况，有利于本文理解 SSD 故障的一些潜在原因。其次，第 2.2 小节介绍了基于机器学习的故障预测技术的实现流程以及其存在的一些挑战，本文基于此流程对预测技术进行了优化，提升了预测能力，同时也基于数据中心常见的 RAID 场景对本节提到的挑战做出了应对设计。再次，第 2.3 小节介绍数据中心中常见的 RAID 机制原理以及其存在的弊端，这有助于本文理解如何优化故障预测技术在实际场景中的部署。最后，第 2.5 小节对本章内容进行了归纳总结。

2.1　SSD 设备基本架构及错误原理

2.1.1　SSD 基本架构

图 2.1 提供了服务器 SSD 架构的图示。大体上来说，一个完整的 SSD 由闪存芯片和 SSD 控制器组成，有些情况下也会包含一个较小的 DRAM 来保证性能和功能。

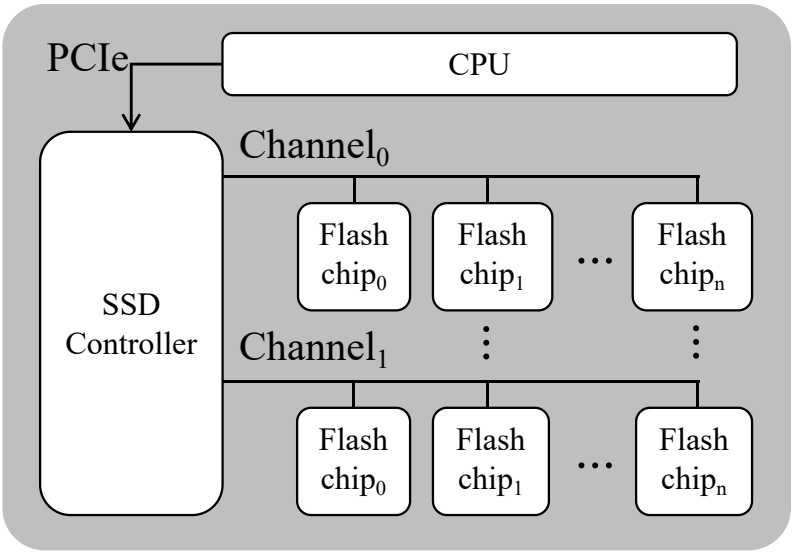


图 2.1　常见 SSD 架构

中国知网　https://www.cnki.net

SSD 会采用多 channel 的形式，每一个 channel 都由 SSD 控制器控制并行运行，同时每个 channel 都会连接多个闪存芯片。尽管这样可能会延长芯片的读写延迟，但是这样的设计显著提升了整体带宽。多个 channel 通过映射数据通道并行提供数据，这有助于确保 SSD 可以在接近 PCIe 连接带宽的情况下为数据提供服务。

SSD 控制器协调与服务器之间的数据传输，负责 SSD 整体的性能情况。除了要管理闪存芯片的访问控制外，SSD 控制器还需要管理闪存芯片的可靠性。这是因为闪存单元在写入次数过多后会磨损，因此 SSD 控制器还需要执行磨损均衡。简单来说就是将写入更均匀地分布在每个闪存芯片中的单元之间，避免反复对同一块单元进行磨损而过早产生读写故障。在磨损均衡的过程中，SSD 控制器必须知道什么时候擦除闪存数据页面（大小约 8KB），以及将该闪存数据页面复制到哪里去，并将需要擦除或复制的页面分组到块中（块约为 128×8KB 页）。为了帮助上层的系统或者应用软件因此闪存数据具体的内部布局，SSD 控制器还需要维护逻辑地址到物理位置的映射，通常被称为是闪存转换层（Flash Translation Layer，简称 FTL）。Chung 等人概述了 SSD 设备操作这一重要部分的组织和操作 [16]。FTL 的数据会被存储在 SSD 中的 DRAM 缓冲区中，由 SSD 控制器管理。使用 DRAM 来存放 SSD 控制器元数据，或者利用 DRAM 作为数据写入闪存芯片的缓冲空间，已经是绝大部分厂商采用的常规设计。我们观察的 SSD 就使用 DRAM 缓冲区空间来存储与逻辑地址到物理地址的 FTL 映射相关的元数据，具体来说会表现在闪存芯片写入量和主机写入量不同。DRAM 高速访问的特性可以允许 SSD 控制器快速定位 SSD 上的数据，从而减少地址转换对性能的影响。

当数据写入 SSD 时，由于 SSD 无法就地更新的特性，可能会触发称之为垃圾回收（Garbage Collection，简称 GC）的操作。页面在这个过程中会被复制，以释放要擦除的未使用数据块，并使得闪存芯片之间的磨损更均匀。然而，每一次擦除都会导致块的磨损 [17, 18, 19]。SSD 控制器会在后台执行 GC 操作，以确保当前的 SSD 有足够的可用空间来写入新数据。

当有一个块被 SSD 控制器认为时不可靠的，并且不应该被进一步使用时，SSD 控制器会选择进行丢弃。丢弃这些块会降低 SSD 的整体容量，从而影响 SSD 的使用寿命，但同时，丢弃块也避免了访问不可靠单元，进而减少 SSD 产生的错误量。

中国知网 https://www.cnki.net

因此，在 SSD 内部这些复杂的操作机制之下，故障影响因素本身也成为了一个十分复杂的综合性问题。

2.1.2　常见的 SSD 错误产生原因

在本文的论述中，会区分 SSD 的故障与错误。通常来说，错误指的是 SSD 的比特翻转，即位错误。这一类错误通常可以被 SSD 上本身存在的错误纠正码（Error Correcting Code，简称 ECC）校验机制修正，能够修正的我们称之为可纠正错误（Correctable Errors，简称 CE），无法修正的称之为不可纠正错误（Uncorrectable Errors，简称 UE），尽管存在 ECC 不可修正的 UE，需要上层机制对其进行修复，但是通常不会对该 SSD 的其他区域访问产生干扰。而故障往往意味着 SSD 的访问出现障碍，甚至出现无法访问的情况，故障产生的原因更为复杂，尽管错误不是故障产生的充分必要条件，但也是其产生的原因之一。而且相比较故障，错误的产生原因相对明确，并且已经存在较多的相关研究 [17, 18, 19, 20, 21, 17, 22]。因此在本节中，简要讨论几种主要的错误产生原因。

对于 SSD 来说，存储数据的元部件是浮栅晶体管，数据以捕获在浮栅晶体管上的电荷形式存储。电荷可以逐渐添加到浮栅晶体管中，直到达到某个饱和点。基于电荷可以逐渐添加到浮栅晶体管的特性，因此可以依据浮栅晶体管电压的测量精度来判断此时这个单元所存储的比特状态。将电压范围分为两部分，即可以判断是 0 或者 1，将电压范围分为四部分，即可以判断是 00、01、10 或是 11，以此类推。与之对应，电压分的越细，状态越多，单位存储数据量也越大，但是出现误差的概率也越大，一次次探测电压的难度也会提升，体现在宏观上就是错误率变高，读写速率变慢。在市面上，以每个单元存储一位为代价，以实现更快的单元电压检测，称为单层单元（Single-Level Cell，简称 SLC）闪存器件。服务器 SSD 通常使用每个单元存储更多位，但感应时间较慢，称为多层单元（Multi-Level Cell，简称 MLC）的闪存。基于 SSD 这样的存储原理，可以将错误的产生大致分为耐久性错误、与温度相关的错误和干扰错误。

中国知网　https://www.cnki.net

耐久性错误

相比较于 HDD，闪存单元在每一次编程和擦除的时候都会变得不可靠，这往往被称为编程/擦除周期（Program/Erase cycle，简称 P/E cycle）。由于 P/E cycle 次数过多，闪存单元开始频繁出现比特翻转，这一类错误被称为耐久性错误。

会出一些这一类独特的耐久性特性，是由 SSD 的工作原理导致的。SSD 存储信息的方式是在浮栅晶体管中添加电荷，依据电荷的数量即电压大小来区分当前存储的数据状态。然而，电荷的逐步冲入可以通过逐步增加电流大小实现，但是电荷的逐步退出却无法做到，因此，一旦浮栅晶体管的电压已经到了较高的水平，其无法再一次性回到较低的电压水平，也就无法表示更低的状态，这也就是 SSD 中常说的无法就地更新的原因。目前解决这种问题的办法是添加一个更大的电流，使得浮栅晶体管被击穿，让所有电荷都跑出去，再重新进行编程来获得低电压的数据状态。然而，这样频繁的电流击穿会逐渐改变浮栅晶体管的物理特性，使其存储电荷的可靠性下降。当浮栅晶体管无法有效存储电荷时，也就意味着数据状态很难被有效的保持，因此会出现这一类独特的耐久性错误。

闪存的耐久性错误通过在高度受控的实验室场景下对单一的芯片或者单元进行监控已经被广泛的研究 [23, 24, 25, 26]。同时也已经存在工作量化了 P/E cycle 对于闪存芯片中各种误差机制的影响 [27, 28, 29, 30]。对于 SLC 闪存来说，通常的可允许 P/E cycle 次数在 100，000 以内；而对于 MLC 来说，次数仅为 3000。因此，在闪存单元上维护 P/E cycle 次数非常重要，通常来说 FTL 会对此进行一定的管理：（1）通过执行磨损均衡来确保没有闪存单元比其他闪存单元磨损得多，（2）通过执行页面离线来主动从群体中移除磨损过多的闪存页，（3）刷新可能随着时间的推移而变得脆弱的单元的内容，以及（4）使用纠错代码来容忍单元故障。

尽管由 FTL 对耐久性做一定的管理，但是越来越高的闪存密度依旧对 P/E cycle 问题带来了挑战。

温度相关错误

由于闪存的浮栅晶体管毕竟是电气元件，因此和其他场效应晶体管一样，浮栅晶体管在温度升高的情况下会产生电荷泄漏的问题。这就是有名的阿伦尼乌斯

中国知网 https://www.cnki.net

效应，当阿伦尼乌斯效应被高温激活时，闪存浮栅晶体管的物理完整性就会逐渐退化，进而导致磨损的加速。同时，高温同样会对 SSD 的结构产生破坏，导线和刻板的不断收缩膨胀也会影响其物理结构，导致器件组件发生物理退化。已经有大量的研究针对温度对 SSD 错误的影响进行了研究 [18, 31, 32]。

现代数据中心的 SSD 内部的温度可以从 30°C 到 70°+C [4]。在数据中心中场景下，SSD 阵列往往排列紧密，这就导致 SSD 的工作环境温度会大幅度上升，因此现代数据中心往往会设置冷却手段，主要以气流的方式为主。长此以往，冷却同样成为数据中心的一大笔开销，因此，探究 SSD 错误的温度依赖性成为现代数据中心中一个有趣的探索领域。

干扰错误

干扰错误简单的说就是一个闪存单元的值，会因为和同一个闪存块中相邻单元的访问而改变。这是由于闪存独特的读取机制导致的。在闪存芯片中，同一块中的单元格有一组位线串联而成。当需要读取特定闪存单元的数据时，由于串联的存在，意味着位线上的其他单元格也必须允许这个读取值通过自己，即让这条位线处于电导通的状态，否则，SSD 控制器无法读取到数据。为了做到这样的效果，这条位线必须用一个较大的电压，一般称作直通电压，来确保位线中的单元格全部被激活。

然而，这样一个较大的电压，对于其他相邻的单元格来说更像是一种 “轻微写入”，因为难免会将一些电荷泄漏到这些单元格的浮栅晶体管中。当短时间内存在大量的读取时，这些相邻单元格收集到的泄漏电荷就会变得足以改变原本的数据状态。因此，这种干扰错误往往对抗性的读取访问模式 [22] 下才会发生。需要注意的是，这种影响在写访问模式中并不常见，因为 SSD 控制器通常会将它们批处理在一起，并存储在一个新的页面中。

中国知网　https://www.cnki.net

2.2 基于机器学习的故障预测技术

2.2.1 故障预测技术的基本流程

为了避免 SSD 故障造成数据丢失，进而产生更为严重的影响，自动检测分析及报告技术（Self-Monitoring Analysis and Report Technology，简称 S.M.A.R.T）应运而生。S.M.A.R.T. 机制能够监控 SSD 运行中多个维度的状态，比如（1）自身属性，通常包含容量、密度、厂商、接口类型等，能够让人迅速了解这块 SSD 的来源。(2）负载属性，比如请求的读取量和写入量，这些数据还区分了主机负载量和实际写入闪存 NAND 芯片的负载量，以此来展示可能存在的 DRAM 缓存带来的影响。(3）环境属性，S.M.A.R.T. 通常会记录 SSD 的工作温度，通过记录的温度变化情况，方便探究 SSD 错误的温度依赖性。(4）错误信息，作为故障的重要相关因素，可纠正错误和不可纠正错误的数量同样会被着重记录。尽管目前还没有一个统一的标准，但是能够被查到的已经有约 180 个不同的属性。除开记录必要的信息，S.M.A.R.T. 通常还会对这些属性设置一个阈值，当数值超过这个阈值时，就会提醒用户，此时的 SSD 健康状况已经出现问题。然而，SSD 的故障已被证实是一个具有挑战性的多维数据问题，使用 S.M.A.R.T. 设置的阈值，无法准确预测 SSD 故障，因此，针对于 SSD 的故障预测通常会依赖机器学习算法进行。机器学习能够让计算机在大量的数据和统计信息中学习到数据未来的趋势，因此十分适合故障预测任务。通常来说，故障预测领域包括离线机器学习算法和在线机器学习算法。

在线机器学习，这种方法要求较高的实时性，这意味着模型可能需要每天更新。此外，为了更敏锐地捕捉负荷变化的影响，在线方法通常不会使用所有历史数据。最新数据通常以滑动窗口的形式进行过滤和训练。这是它与离线学习最大的不同之处。此外，由于在线学习方法使用的数据量较小，机器学习算法的选择也会有所不同。在线学习更倾向于离群点检测，通过统计学习找出与大部分数据不同的离群点。相比之下，离线学习更像是分类任务，将健康设备和故障设备视为两类不同的设备。因此，在线学习更适合工作负载多变的场景。但丢弃历史数据的做法不可避免地会导致性能下降。

而离线方法无需实时更新。相反，只需每隔一段时间使用不断增加的数据集

中国知网 https://www.cnki.net

重新学习模型。这意味着离线机器学习算法更适用于已有一定数量数据集且工作量不会经常变化的情况。这非常适合当前的数据中心场景，也是目前被使用较多的方法。本论文中讨论并改进的均为离线学习。

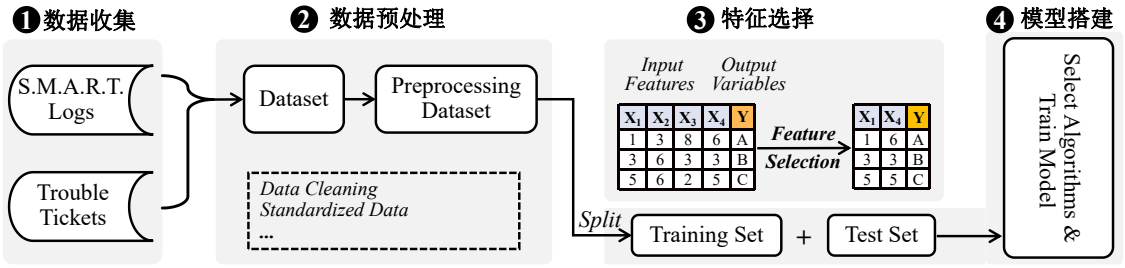


图 2.2　基于机器学习的 SSD 故障预测常见流程

使用离线机器学习算法预测固态硬盘故障的过程包括四个主要步骤，如图2.2所示。这四个步骤分别是数据收集、数据预处理、特征选择和模型构建。

数据收集

首先，数据收集过程是收集日志信息，以反映设备的状态（即故障或健康）。数据收集的主要来源是长期记录的 S.M.A.R.T. 日志（步骤❶），以及数据中心运维人员保留的设备故障单（即故障时间、故障位置）。

数据预处理

其次，数据预处理是指对收集到的数据集进行修改或完善，以方便后续的模型学习，其中包括数据清理、标准化数据以及其他一些专门的数据集处理（步骤❷）。良好的数据预处理可以使数据更加整洁，并消除任何异常。这将改善模型的性能，提高模型的可靠性。因此，也有人在研究如何更好地进行数据预处理 [13]。

特征选择

第三，处理后的数据集通常格式整齐，有输入特征和输出变量。但并不是所有的输入特征都适合作为模型训练的数据，这是因为与故障无关的属性同样可能存在分布差异，从而影响模型的判断。特征选择过程旨在剔除不相关的属性，以提高模型性能并减少模型训练时间（步骤❸）。如图2.2所示，经过特征选择后，从四个

中国知网　https://www.cnki.net

属性 $\{X_1, X_2, X_3, X_4\}$ 中选出与最终分类类别 Y 相关的两个属性 $\{X_1, X_2\}$ 用于后续模型的建立。

模型建立

最后，建立模型的过程是将数据集分成训练集和测试集。训练集用于模型学习，测试集用于验证模型的性能。然后，选择合适的机器学习算法进行训练和验证(步骤❹)。

2.2.2 预测模型的挑战

对于预测模型的性能，通常来说会有两个较为重要的指标:

- **Precision Rate (PRE)**: 正确预测故障的设备数量与预测为故障设备数量之比。
- **Recall Rate (RECALL)**: 正确预测故障的设备数量与实际故障设备总数之比。

对于这两个指标来说，都是越高越好。PRE 越高，意味着预测越准，出现将健康设备预测为故障设备的次数越低；RECALL 越高，意味着预测越全，出现将故障设备预测为健康设备的次数越低。然而，对于训练有素的模型来说，这两个指标之间通常存在着折衷，即提高一个指标通常会降低另一个指标。这是不可避免的，如图 2.3所示。对于故障预测任务来说，可以将其认为是一个本质为分类的任务，即分类为会发生故障和不会发生故障。然而由于实际生产环境的复杂性，难免存在故障设备的 S.M.A.R.T. 信息和健康设备接近或是健康设备的 S.M.A.R.T. 信息和故障设备接近。在图中，简化为单维度便于理解。可以看到，故障设备和健康设备大部分是区分开的，但是难免存在少部分的交叉，而模型的任务就是设置一条阈值将这两类区分开来。因此，为了达到高 PRE，难免会将实际故障设备标记为健康；反之，高 RECALL 会将健康设备标记为故障。因此，通常来说，对于一般的场景会追求一个较为平均的方式让二者的水平相对较高。

但是在数据中心针对于故障 SSD 的预测场景下，需要考虑经济效应。实际的使用中会偏向于高 PRE，即希望尽可能的减少将健康设备预测为故障的情况。因为相比较于市面上较为廉价的消费级 SSD，数据中心中使用的企业级 SSD 价格往

中国知网 https://www.cnki.net

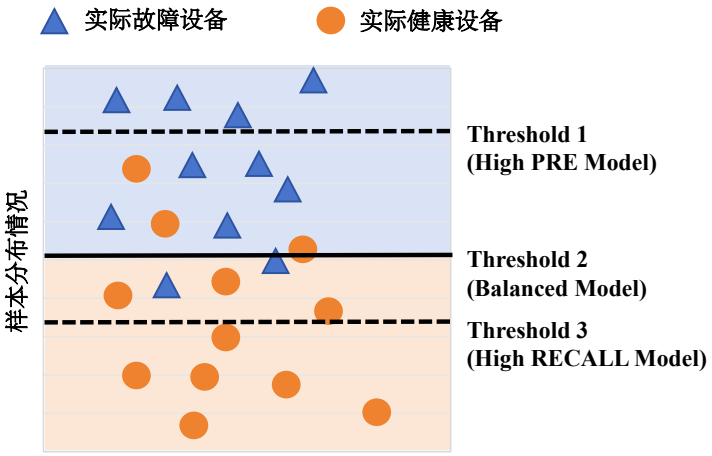


图 2.3　PRE 和 RECALL 折衷示意图

往会高出十余倍，同时容量也会远大于消费级 SSD。而高 PRE 可以很好的避免经济上和容量上的浪费。但这样的代价就是 RECALL 的降低，能够成功识别出来的故障设备变少，需要使用上层冗余机制来恢复数据的情况改善有限。因此，如何改善预测模型的实际部署方式也是一个值得探讨的挑战。

2.3　基于奇偶校验的 RAID 冗余技术

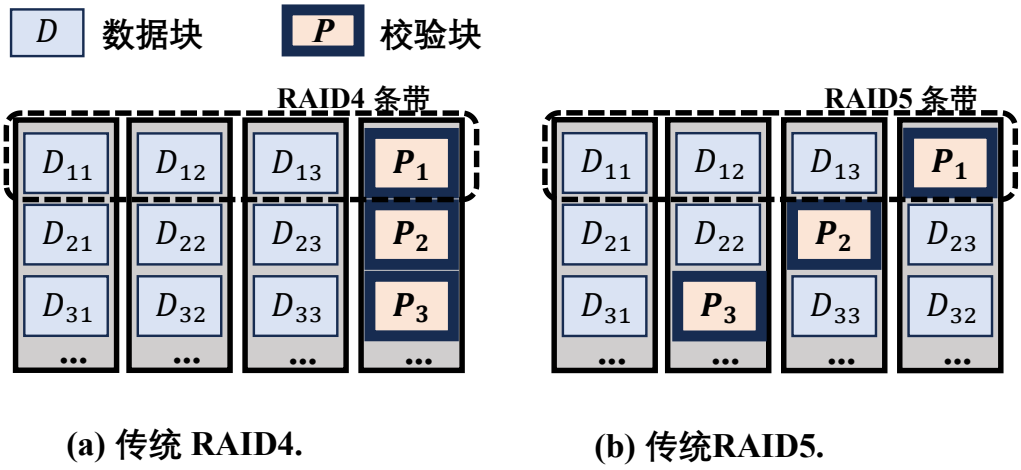
基于 HDD 和 SSD 可能存在的可靠性问题。数据中心往往会在上级设置冗余或者校验机制来保证服务的稳定性。磁盘阵列（Redundant Arrays of Independent Disks，简称 RAID）被广泛的使用。RAID 为大规模数据存储提供了很多便利：（1）数据保护，现代的 RAID 基本都带有校验机制，校验机制的存在可以保证一个 RAID 阵列中存在一块设备出现故障也不会影响整体的数据完整性。（2）传输速率的提升，RAID 独特的条带化机制，极大利用了这些设备的并行能力，使得数据吞吐量可以明显提升。

2.3.1　RAID 机制的基本原理

基于奇偶校验的 RAID 系统以较低的成本和复杂性提供存储设备故障保护能力。典型的基于奇偶校验的 RAID 方法包括 RAID4 和 RAID5。如图 2.4所示，即为传统的 RAID4 和 RAID5 架构。图中的两个 RAID 架构是以四块设备为例建立的，

中国知网　https://www.cnki.net

一般来说，对于这类基于奇偶校验的 RAID 只需要超过两个设备均可。但是设备数量越多，保护力度越小，因此视具体情况而定。基于奇偶校验的 RAID 系统的存储空间以条带为单位组织。每个条带由块组成，条带中的每个块来自阵列中的不同存储设备。可以看到，条带中的块分为两类，一类是数据块，这种块用于存放数据，还有一类是校验块，用于实现校验机制保护数据。



(a) 传统 RAID4. **(b) 传统RAID5.**

图 2.4 RAID4 及 RAID5 架构

和存放数据的数据块不同，作为冗余的奇偶校验块是通过对用户数据块执行 XOR 操作生成的。即图中的 P_1 是由 $\{D_{11}, D_{12}, D_{13}\}$ 执行 XOR 操作生成。RAID4 和 RAID5 的差别在于 RAID4 会将一块设备单独用来存放所有校验块，而 RAID5 则是将校验块均匀分布在所有设备中，避免了 RAID4 可能出现的单点故障问题。同时，将奇偶校验块分布在阵列中的所有设备上，可以有效分散 I/O 压力，避免 RAID4 中校验设备成为性能瓶颈的情况。这两种架构都可以承受一个设备故障，而不会丢失数据。

基于这样的架构，恢复丢失的用户数据块就变得非常简单。以图中的 P_1 所在的条带为例。假设 D_{11} 所在的设备出现故障，无法获得其存放的数据，此时只需要计算 $\{P_1, D_{12}, D_{13}\}$ 这三者的 XOR 操作结果即可，这个操作结果就是 D_{11} 所存放的数据值。这是由 XOR 计算的自反性决定的。

中国知网 https://www.cnki.net

2.3.2　常见 RAID 架构

除了上个章节介绍的 RAID4 和 RAID5，还有许多其他形态的 RAID 架构，这些架构有着各自的优点和缺陷，接下来将会对这些 RAID 架构进行简要说明。

RAID 0

RAID0 是一种比较简单、不存在校验机制的条带化架构。从架构上看，RAID0 甚至很难被称为真正的 RAID 架构，因为其根本就不提供数据的冗余服务。如图2.5（a）所示，RAID0 会将数据进行条带化，将数据分散的放在所有的设备中，这样就可以对每一个设备进行单独读写以实现数据的并行访问。由于 RAID0 可以实现并行访问，同时不存在数据校验机制，因此理论上讲，RAID0 是所有 RAID 中性能最好的，一个由 n 个存储设备组成的 RAID0 ，它的读写性能是单个存储设备的 n 倍。

RAID0 的成本很低，其存储空间能够被完全利用，但是却有着很高的读写性能。同时，完全利用的存储空间也意味着 RAID0 无法提供数据保护，当其中的设备发生故障时，系统将无法正常工作。因此，RAID0 无法在可靠性要求很高的场景下使用，一般情况下，一些音视频场景可以使用 RAID0 来利用其较高的读写性能。

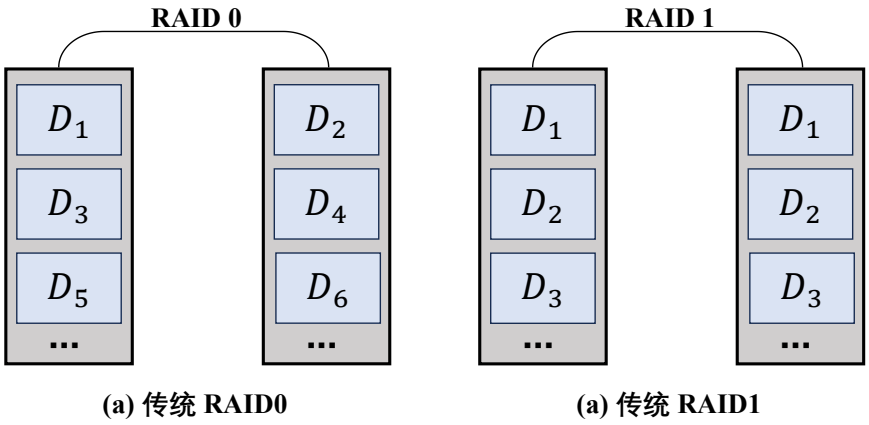


图 2.5　传统 RAID0 及 RAID1 架构

中国知网　https://www.cnki.net

RAID 1

如图2.5（b）所示，RAID1 和 RAID0 不同，RAID 不存在数据条带，但是却存在数据冗余。RAID1 其实可以理解为就是进行了一次完备的镜像，通过设置一个镜像的存储设备，存放于工作存储设备完全相同的数据来保证数据的可靠性。从原理来看，RAID1 中其实只有一半的空间是存放的有效工作数据。利用拷贝的方式，RAID1 能够在其中一块存储设备发生故障时，利用另一块设备进行无障碍的访问。这种实现成本较高，但是数据保护能力强的架构就会用在一些安全要求高的场景，比如银行和邮件系统。

RAID 6

前面章节所介绍过的 RAID4 和 RAID5 尽管都存在数据校验机制，能够在存储设备出现故障时恢复数据。但是却都无法承受两个存储设备同时发生故障。相比较之下，如图2.6所示，RAID6 解决了这个问题。RAID6 不再使用单一的校验块，而是使用两块校验块，这样，即使阵列中同时出现两个存储设备无法访问，整个系统都可以继续运作。在某种层面上来说，RAID6 可以看作是 RAID4 和 RAID5 的可靠性加强版本。

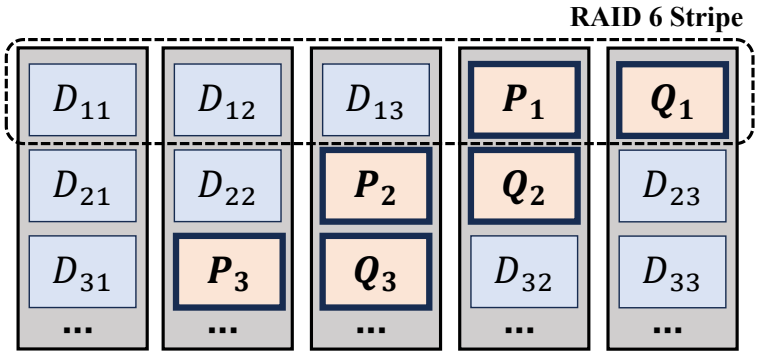


图 2.6 传统 RAID6 架构

需要注意的是 RAID6 中的两个校验块往往并不是简单的拷贝保存，一般来说，是采用两个完全不同的校验算法，如图2.6所示，生成校验块 P 和校验块 Q。同时，校验块 P 和校验块 Q 和 RAID5 一样分散在不同的存储设备上。当出现两个设备同时发生故障时，恢复数据就可以看成是求解两元方程。

中国知网 https://www.cnki.net

尽管 RAID6 提供了更强的数据保护能力，但是从其恢复数据原理就可以看出，其控制器更为复杂，且冗余的校验数据更多，写的性能也会因为要生成更多校验数据而变差，因此成本更大。只能用在对数据安全要求特别高的场景。

RAID 0+1

RAID0+1 的基本原理是先进行 RAID0，然后再将多个 RAID0 阵列组合成 RAID1，如图2.7（a）。

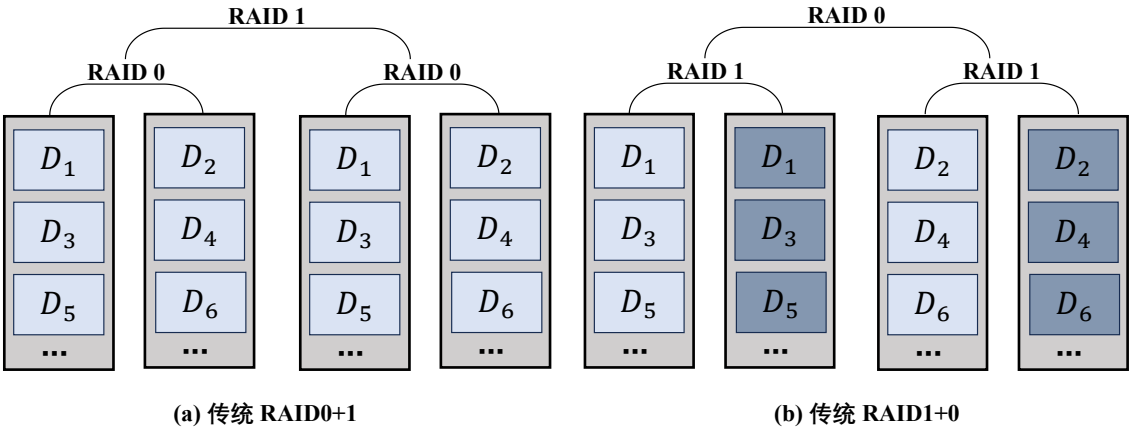


图 2.7　传统 RAID1+0 和 RAID0+1 架构

使用 RAID0+1 的原因是为了进一步增加数据保护能力。以图2.7（a）为例，当第一块存储设备和第二块存储设备都发生故障时，整个系统依旧能够继续运行。然而当第一块存储设备和第三个存储设备或者第一块存储设备和第四个存储设备同时发生故障时，系统就会发生崩溃。这是因为对于 RAID0 来说，这是一个整体的数据条带系统，有一块设备发生故障就会导致系统崩溃。尽管，RAID0+1 进一步增强了数据保护能力，但是对于保护两块设备同时发生故障的情况依旧限制较大，且代价昂贵，因此被使用较少。

RAID 1+0

RAID1+0 的基本原理是先进行 RAID1，然后再将多个 RAID1 阵列组合成 RAID0，如图2.7（b）。

和 RAID0+1 一样，RAID1+0 同样为了增加数据保护能力。但是 RAID1+0 的

中国知网　https://www.cnki.net

保护能力更强，以图2.7（b）为例，只有当第一块存储设备和第二块存储设备或者是第三块存储设备和第四块存储设备同时发生故障时，才会导致整个系统的崩溃。这是因为 RAID1 作为镜像设备并不会因为一块设备的故障而发生崩溃。由于 RAID1+0 拥有更强的数据保护能力，适用于读写性能要求高，数据安全大于存储成本的场景。如银行、数据库等。

2.4 本章小结

本章节给出了本研究使用的技术背景及需要解决问题的已有解决方案。首先主要介绍了 SSD 的一些物理原理以及架构情况，进而引出 SSD 本身如何出现错误。其次，介绍了针对数据中心 SSD 故障这个问题目前业界正在研究的解决方案，即利用机器学习方法来进行预测，进而对数据进行提前保护，保障上层应用服务的稳定性。本章节中详细介绍了基于机器学习的故障预测目前常见的流程以及存在的挑战。最后，针对于数据中心常见的应用场景即 RAID，介绍了其实现原理以及常见的架构类别。

中国知网 https://www.cnki.net

第三章　数据分析：数据中心故障 SSD 的运维数据分析

为了进一步了解数据中心这类集群 SSD 场景中，故障 SSD 和健康 SSD 之间的差异，以更好的改善故障预测手段。本章节对真实数据中心的运维数据进行了全面的评估和深入的分析。第 3.1 小节介绍了本文使用的数据集来源以及其包含的具体信息情况并按照 NAND 类型、容量对 SSD 进行了区分以减少对比分析时变量的影响。同时，展示了不同类别中故障 SSD 在时间上的变化情况，并获得了清晰的“早期死亡率”现象。第 3.2 小节更为详细的针对于 S.M.A.R.T.，从不同角度对故障设备所表现出的数据趋势差异进行了统计分析。第 3.3 小节基于前述分析所得的结论，提出了一个全新的概念，即 SSD “状态” 概念，旨在成为区分潜在故障原因的启发性方法。最后，第 3.5 小节对本章内容进行了归纳总结。

3.1　数据集来源及基本信息

为了能够更好的了解实际集群规模 SSD 的故障特性。本文从华为的数据中心收集数据。数据中心按照 model 进行分布。每个 model 中都由多个 device 组成。每个 device 都代表存放盘片的设备号。同时每个 device 中存放了一个或多个盘片，每个盘片都有盘片序列号（每个盘片唯一)。本文得到的数据集最早可以追溯到 2017 年 10 月，最迟到 2021 年 9 月，覆盖了超过 20 万个 SSD 设备。值得注意的是，在本文的观察中，每个 model 中的 SSD 并不统一，在供应商、容量、部署时间、规格上存在异构性。但是每个 device 似乎对应了数据中心的采购批次，其中的 SSD 规格较为统一。如此大量的数据使得本文后续的分析拥有实际统计意义。

3.1.1　SSD 类别区分

具体 SSD 分类如表 3.1所示，数据集中包含 MLC 和 TLC 闪存。对于每种 NAND 类型，这些基于闪存的固态硬盘可根据固态硬盘容量进一步分为三类。因此，我们可以根据固态硬盘的大小得出六个类别，即 MLC 闪存的 A1-A3 和 TLC 闪存的 B1-B3。需要注意的是，这六种模型的制程和预留空间（overprovision，简称 OP）

中国知网　https://www.cnki.net

表 3.1　SSD 分类信息

类别	NAND 类型	容量 (GB)	数量	总占比 (%)
A1	MLC	600 ~ 960	20521	11.2%
A2		1600 ~ 1920	5019	2.7%
A3		3200 ~ 7680	9730	5.3%
B1	TLC	600 ~ 960	35594	19.5%
B2		1600 ~ 1920	44108	24.1%
B3		3200 ~ 7680	67011	36.7%

以及产品等级都是相同的，均为 15mm 的企业级 SSD。同时，由于数据集中的 SSD 容量类型过于丰富，导致划分不便，因此将相近容量的设备划为一类。此外，我们还可以计算每个类别的固态硬盘数量（固态硬盘数量）和占固态硬盘总数的比例（比例（例如，类别 A1 代表使用 MLC 闪存的硬盘，容量从 600GB 到 960GB。A1 类数据集包括 20521 块固态硬盘，占固态硬盘总数的 11.2 下面，我们将使用这六个类别进行故障研究和讨论。

3.1.2　S.M.A.R.T. 内容介绍

先前提到过，S.M.A.R.T. 包括许多方面，在 HDD 的 S.M.A.R.T. 标准规范中包含 100 多种不同类型的指标，尽管 SSD 目前还没有相关统一标准，但是同样类型繁多。但在实践中，不同的供应商会为不同外形尺寸的设备提供不同数量和类型的 S.M.A.R.T. 属性。本论文使用的数据集包含了在四年的时间跨度内收集到的 SMART 日志，其收集到的 SMART 属性汇总在表 3.2中。值得注意的是，由于时间跨度较大，收集到的数据并不完整，存在缺失的情况，即并非每天的数据都能够收集到，但是最终依旧能够收集到超过三百万 drive days，这使得本文的统计分析依旧存在实际意义。同时，由于不同供应商对 SMART 属性的定义有所不同，为了便于比较，本文将 SMART 属性按其监视类型分为四组，即固有属性、错误相关属性、工作负载相关属性和持续时间及磨损属性。

中国知网　https://www.cnki.net

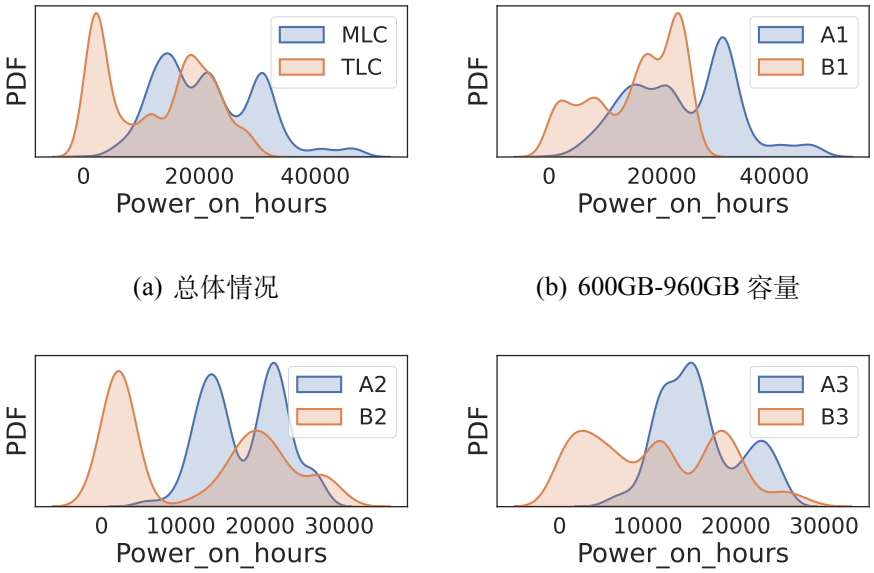
表 3.2　本文关注的 S.M.A.R.T. 信息列表

(a) 固有属性	
属性	**含义**
Capacity	设备容量
NAND_type	NAND 闪存类型
Die_density	Die 密度 (NAND 特性)
Fbb	出厂坏块数
(b) 错误相关属性	
属性	**含义**
Gbb	新增坏块数
Nonmedia_error	非介质错误计数
Unc	不可纠正错误
Esf	要擦除的失效块数
Psf	编程失效的区段数
(c) 工作负载相关属性	
属性	**含义**
Host_read_byte	主机端读取量 (bit)
Host_write_byte	主机端写入量 (bit)
NAND_read_byte	NAND 闪存读取量 (bit)
NAND_write_byte	NAND 闪存写入量 (bit)
Wa	写放大系数
Read_write_percent	读写比
Dwpd_real	设备平均每天写入量
(d) 持续时间及磨损属性	
属性	**含义**
Power_on_hours (Poh)	通电小时数
Life_percentage_used (Lpu)	（编程/擦除）次数

中国知网　https://www.cnki.net

3.1.3　基于时间的故障分布情况

为了了解固态硬盘故障的总体分布情况，本文根据 Poh（开机时间）属性对数据集进行了统计分析。图 3.1显示了故障在时间跨度上的概率分布函数。从图中可以看到两个十分明显的差异。



(a) 总体情况　(b) 600GB-960GB 容量

(c) 1600GB-1920GB 容量　(d) 3200GB-7680GB 容量

图 3.1　各类型 SSD 的故障在时间跨度上的分布

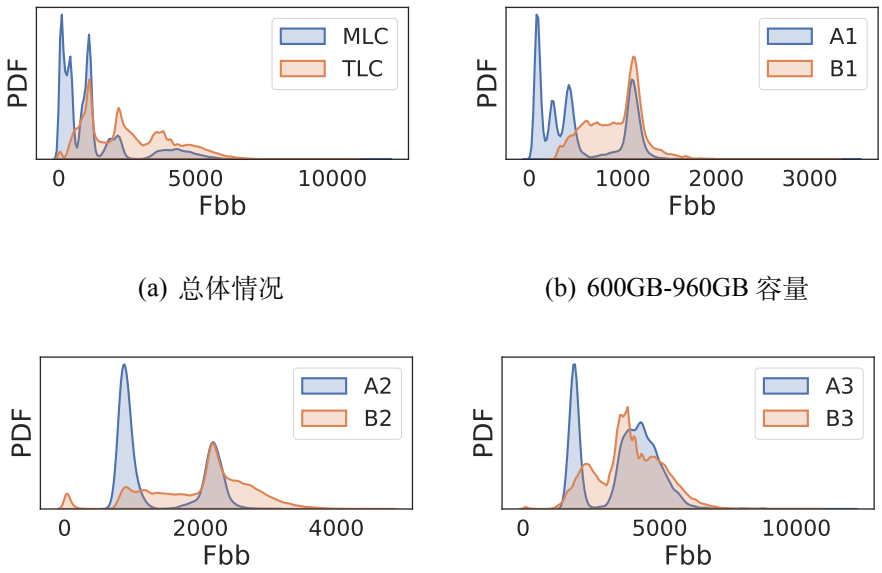
首先，MLC 和 TLC 闪存的故障分布存在显著差异。如图（a）所示，总体而言，TLC 故障分布的形状与 MLC 类似，但是总会比 MLC 更提前一点，不仅是整体情况，即使在不同的容量分类范围下，都呈现类似情况（图 b）-（d））。其次，TLC 和 MLC 故障分布的形状和日常生活中的经验较为相悖。日常生活中的经验总会认为设备用的越久越容易坏，而这也和传统可靠性理论中常用的“浴缸”模型类似[33, 34, 35]，“浴缸”模型中假定故障率会随着时间增长变得越来越高。然而在本文的观察中，故障的分布更为复杂一些，呈现出一种先增长再下降再增长的趋势，尤其是早期增长的“早期死亡率”现象十分值得分析。

中国知网　https://www.cnki.net

3.2　基于 S.M.A.R.T. 信息的故障差异分析

为了解释先前观察到的两个现象，本文利用收集到的 S.M.A.R.T. 信息进行了针对性的数据分析。

3.2.1　MLC & TLC 差异



(a) 总体情况　　(b) 600GB-960GB 容量

(c) 1600GB-1920GB 容量　　(d) 3200GB-7680GB 容量

图 3.2　不同 SSD 类型的出场坏块数分布

出厂质量

从设计原理来看，TLC 固态硬盘由于需要更精细地识别阈值电压。对于 MLC 来说，一个浮栅晶体管存储 2 个 bit 的信息，因此需要识别 4 个逻辑电平，而 TLC 则是一个浮栅晶体管存储 3 个 bit 的信息，这意味着 TLC 有总共 8 个逻辑电平，电压需要更加精确以将电荷存储在每个逻辑电平所需的窗口内。这在减慢了 TLC 的编程速度的同时，也给浮栅带来了更大的负担，使得 TLC 的可靠性下降，尤其是 3D 结构容易造成读写间的干扰，进一步影响 SSD 的可靠性。因此由于 TLC 出厂时的可靠性不足导致更早的故障发生是一个合理的猜测。

中国知网　https://www.cnki.net

为了验证这一猜测，本文对 Fbbs（出厂坏块）进行了统计分析，以探讨这种设计差异是否会导致数据中心应用场景中的故障分布差异。S.M.A.R.T. 属性中的 Fbbs 可以反映固态硬盘刚部署时的可靠性，也与固态硬盘的质量有关。图 3.2展示了故障设备中 TLC 和 MLC 的 Fbb 分布差异。从结果来看，作为故障设备，TLC 的 Fbbs 多于 MLC。这验证了故障原因的差异确实可能反映在单个属性上。

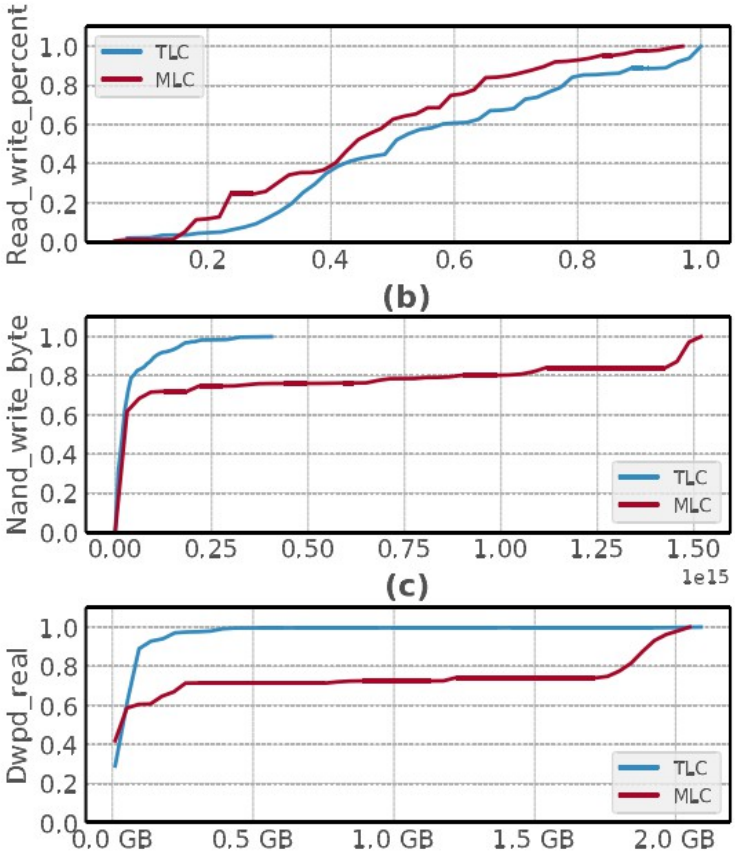
工作负载

除了出厂质量，本文还关注 MLC 和 TLC 之间的工作负载差异。这实际上触及了一个非常有趣的话题，即数据中心采购固态硬盘的目的。在以前的工作中，所有设备都被认为用于相同的目的。这是数据中心整个复杂而庞大的数据调度系统造成的，而不是人为地希望某些设备用于某些特定的应用场景。然而，实际情况中，不同设备之间的工作负载差异很大。数据中心的设备采购量往往很大，这就导致设备采购的时机很少是为了弥补少数坏设备缺失的离散情况。更常见的情况是，由于数据中心业务的增加而购买新的设备，而业务的增加有时会伴随着新业务的产生。对于固态硬盘来说，这就是新的工作负载，这会导致不同批次固态硬盘所处的应用环境出现整体差异。这反过来又会影响固态硬盘的故障性能。回到本文的数据集，从机架等位置信息观察到，同一类型的固态硬盘的集中度相当高。它们的部署时间相对一致，这意味着 MLC 和 TLC 之间可能存在差异。

然而，系统工作量包括敏感信息，也很难收集。作为高校只能通过一些 S.M.A.R.T. 数据来确定固态硬盘的工作量。本文列出了 TLC 和 MLC 的一些主要工作量指标，包括主机读写、NAND 读写、读写比率和 DWPD（驱动器每天写入次数）参数。一些更显著的差异得以图表的形式显示出来。

如图 3.3所示，该图的横坐标代表了当前属性的值分布，而横坐标则代表了值分布对应的累计分布。在本文的观察中，MLC 和 TLC 的读取情况几乎相同，但是二者的写入情况却有一定的变化。首先可以明显观察到 MLC 所经历的写入更多，这和 MLC 本身的特性相符合，因为 MLC 确实比 TLC 更加适合写入操作，这是由内部的实现特性决定的。这说明数据中心确实有意的使得设备运行在更加适合自己的场景之下，以此获得更大的效益。其次，相比较于先前的研究 [36]，本文观察

中国知网　https://www.cnki.net



(c) 设备平均每天写入量

图 3.3　MLC 与 TLC 在三种工作负载属性上的累积分布差异

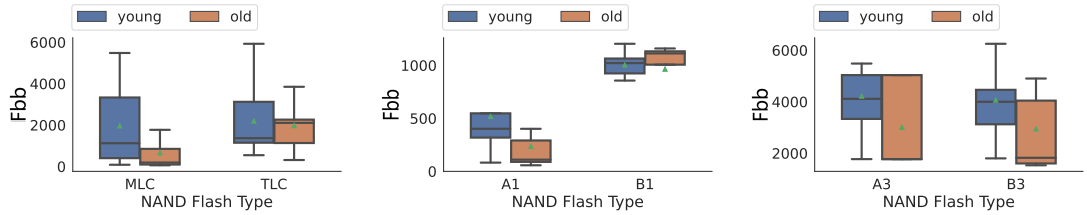
到了一个看上去比较“另类”的数据，在我们数据集对应的一个纯 flash 系统中，观察到了始终 <1 的读写比，TLC 的均值为 0.5，MLC 的均值为 0.55。也就是说，这个纯 flash 的系统运行在写密集型环境之下，说明现在的闪存集群已经被用在更多的场景之下，针对于闪存集群的故障特性需要我们开始考虑更多的可能性。

3.2.2　早期死亡率

通常来说，对于”早期死亡率”的普遍解释是”双池”模型 [4]。这意味着，当一批固态硬盘首次部署时，可以假设它们的出厂质量参差不齐，此时可以逻辑的划分为两个池，即高质量池和低质量池，对于每一块固态硬盘来说，都对应其中的一个池。低质量池中的固态硬盘出厂质量差，消耗很快，因此在部署初期故障会迅速激增。然后，在低质量池中的固态硬盘消耗殆尽后，整体固态硬盘故障水平开始稳定在较低水平。随着部署时间的延长，高质量池中的固态硬盘逐渐耗尽。因此在

中国知网　https://www.cnki.net

最终，在部署时间较长的阶段，固态硬盘故障率又开始上升。



(a) 总体情况　(b) 600GB-960GB 容量　(c) 3200GB-7680GB 容量

图 3.4　Fbb 数值在设备不同年龄阶段的分布。条形图的水平线为中位值，绿色三角形为平均值

为了验证”双池”模型，本文使用了 Fbb 属性进行进一步分析，如图 3.4所示。所有发生故障的固态硬盘都根据 Poh，即通电时长，分成两组，即“年轻”组和“年老”组。相比较于固态硬盘的出厂时长，因为存在负载迁移、业务调整的缘故，可能存在较长时间的停机，所以通电时长会更加符合固态硬盘的寿命一说。需要注意的是，A2 和 B2 类别的故障样本数量太少，无法进行分析，因此本文只展示了 A1 和 B1 类别、A3 和 B3 类别的结果。为确保数据量的一致性，本文在分类中使用的阈值取自第 50% 次部署时间。条形图代表着该情况下的数值分布，以如图 3.4（a）所示，可以看到“年轻”MLC 的 FBB 数值分布在 200 ~ 3800 左右，而平均值在 2000 左右，相对应的“年老”的 MLC 数值分布仅为 100 ~ 500，且平均数仅为 400 左右。这是一个非常有趣的现象，当统计所有发生故障的固态硬盘的 Fbb 属性时，较年轻的 MLC 的数据分布范围明显更广，中位数和平均值也远高于较年长的 MLC，中位数为 5.96 倍，平均值为 2.99 倍。在 TLC 中，情况并没有那么明显，尽管中位数为 ×0.65，平均数还是达到了 ×1.11。很容易可以得出几个结论。首先，与较老的 MLC 闪存相比，较年轻的 MLC 闪存的数据分布范围更广，中位数和平均值也更高。TLC 闪存的分布相对集中。其次，通过对其余两个类别（A1 和 A3）的比较，我们可以发现，在小容量和大容量固态硬盘中，较年轻的故障设备的 Fbb 数量要多于较老的故障设备。

为了进一步验证分布差异，本文使用双样本 Kolmogorov-Smirnov（KS）[37] 检验来统计评估“年轻”和“年长”SSD 之间 Fbb 分布的差异（置信水平为 95%）。

中国知网　https://www.cnki.net

最终得到的 P 值均远小于 0.05，拒绝了原假设，支持了备择假设，表明两者的分布差异显著。结合之前的通电时长分布图，本文将 TLC 闪存的影响不太明显的原因归结为目前 TLC 闪存的整体年龄较小，以及 TLC 更偏重于读取的工作负载所造成的差异。

3.2.3　一些其他差异及总结

除上述两个关键属性外，还存在与上述故障相关属性内部变化情况类似的其他属性。例如，Unc（无法纠正的错误）是一个具有代表性的指标，尤其是在 MLC 闪存中。在发生过 Unc 的设备中，Gbb（新损坏块的数量）达到了 32.4%，而在从未发生过 Unc 的设备中，Gbb 的百分比仅为 6.3%。两者相差 5 倍。虽然 Gbb 与 SSD 故障之间没有直接联系，但有 Unc 和没有 Unc 的设备之间已经存在显著差异。

通过对于闪存类型、工作负载及其他 S.M.A.R.T. 属性的分类对比分析，可以发现故障原因的差异可能反映在某些 S.M.A.R.T. 属性的数值分布差异上。比如对于 NAND 类型，不同的 NAND 类型由于工作负载以及本身可靠性的差异会与最终的故障原因产生影响，比如 MLC 可能会偏向于高强度写入操作所带来的磨损原因，而 TLC 则可能其自身坏块导致的可靠性问题会有较大影响。又或者是通电时长，在不同的通电时长范围内，刚刚加入集群的设备更可能因为出厂质量的原因而发生故障，而已经加入集群很久的设备则更偏向于长时间的使用而磨损导致故障。因此，故障原因的差异可能潜在的反映在某些 S.M.A.R.T. 属性分布差异上。这意味着，可以尝试使用这种方法，在预测故障之前就隔离潜在的故障差异。这对后续的故障预测有着极大的帮助，因为故障预测最大的难点就在于数据杂乱，各种故障原因的表现情况各不相同却有着同样的标签结果，这对模型的干扰极大。

3.3　基于 S.M.A.R.T. 属性差异的 SSD“状态”定义

受属性值与固态硬盘故障之间关联的启发，本文为已部署的固态硬盘定义了一个新概念” 状态”，以更好地利用属性值进行故障预测。因为要区别属性分布差异，因此，本文提出的状态从本质上来讲就是一组序列，这个序列中的各个值对应的是各个属性的分布区间。需要注意的是，“状态”旨在成为区分潜在故障原因的

中国知网　https://www.cnki.net

表 3.3　“状态” 定义

(a) 属性值分割				
	属性值范围			
值区间	I_1	I_2	...	I_n
(b) “状态” difference				
属性 / **“状态”**	$\boldsymbol{A_1}$	$\boldsymbol{A_2}$	...	$\boldsymbol{A_m}$
“状态” 1	I_{A_1}	I_{A_2}	...	I_{A_m}
“状态” 2	I_{A_1}	I_{A_2}	...	I_{A_m}
...	...	...	...	...
“状态” n	I_{A_1}	I_{A_2}	...	I_{A_m}
Note: $I_{A_i} \in A_i\{\boldsymbol{I_1}, \boldsymbol{I_2}, ..., \boldsymbol{I_n}\}$, i∈ [1,n] $\forall x, y \in$ “$States$”, $\exists x(I_{A_i}) \neq y(I_{A_i})$, $i \in$ [1,n]				

表 (a) 展示了数据集的分布按属性值范围分为 n 值区间
表 (b) 说明了 “状态” 的具体表述以及对 “状态” 定义的限制。

启发性方法，因此在生成” 状态” 时，本文只关注故障样本。为了更好的说明 “状态” 的含义，接下来用生成步骤以及一些定义时的限制来展示这个全新的概念。

如表 3.3所示，“状态” 的生成过程如下：

（1） 依据先前的发现，本文的目标是为了发现潜在的反映在某些 S.M.A.R.T. 属性分布差异上的故障原因差异，因此这些属性必须是要和故障本身息息相关。所以生成 “状态” 的首要步骤就是要将一些与固态硬盘故障相关的重要 S.M.A.R.T. 属性选择出来。为了更好的说明，如表 3.3（b）所示，假设此时筛选出 m 个属性，可以表示为：$\{A_1, A_2, ..., A_m\}$。

（2） 第二步即是对筛选出来的属性进行分布差异的区分。如表 3.3（a）所示，对于每个属性，其值范围可在所有故障固态硬盘中划分为 n 个值区间（即：$\{I_1, I_2, ..., I_n\}$）。其中，对于 n 的取值会在第四章中进行说明。

（3） 最后是要对 “状态” 之间的区别做出定义限制。为了体现差异，每一个 “状态” 之间必然需要出现差异。本文认为，对于两个” 状态” 来说，只要有一个值落在某些属性的不同值区间内，就可以认为它们是不同的。例如，如表 3.3（b）所

中国知网　https://www.cnki.net

示，对于“状态 1”这个序列中的第一个值 I_{A_1} 来说，它的取值范围来自于属性 A_1 的属性值范围，即对应属性 A_1 的 $\{I_1, I_2, ..., I_n\}$ 中的一个。同样的，对于“状态 2”来说也一样，但是只要“状态 1”和“状态 2”的 I_{A_1} 不同，哪怕后续的 $\{I_{A_2}, I_{A_3}, ..., I_{A_n}\}$ 都相同，也需要将其视为两个不同的“状态”。

通过这样的三个步骤，就可以将所有数据集中的 SSD 对应要其所对应的一个“状态”当中，这样的方式不仅可以减少数据集的混乱程度，同样也能潜在的区分一些故障原因。对于“状态”设置的进一步讨论和实现，将会在下一章中进行详细讨论。

3.4　本章小结

本章首先研究了华为真实数据中心中收集到的 S.M.A.R.T. 数据集，介绍了数据集的基本情况，并且通过 NAND 类型、容量、数量等合理的将接近 200000 块 SSD 分成了六类。同时，详细介绍了收集到的数据集中涉及的属性类型，依据针对性分成了四类，即固有属性、错误相关属性、工作负载相关属性和持续时间及磨损属性。

其次重点对收集到的数据集进行了分析研究，通过对于闪存类型、工作负载及其他 S.M.A.R.T. 属性的分类对比分析，可以发现故障原因的差异可能反映在某些 S.M.A.R.T. 属性的数值分布差异上。进而提出了“状态”概念，同时详细介绍了状态的生成过程。状态从本质上来讲就是一组序列，这个序列中的各个值对应的是各个属性的分布区间。状态区分的方式不仅可以减少数据集的混乱程度，同样也能潜在的区分一些故障原因。这为后续的数据预处理研究提供基础。

中国知网　https://www.cnki.net

中国知网 https://www.cnki.net

第四章　性能优化：差异化的机器学习 SSD 故障预测研究

4.1　研究动机：数据中心 SSD 故障预测性能不佳

尽管目前已经有较多的工作开始研究数据中心这类集群 SSD 场景下的故障预测，但是目前的效果依旧不佳。以阿里巴巴近期的工作中展示的性能数据为例，如表4.1所示。表中提到的 PM1733 Tencent、PM9A3 Tencent 以及 MB1 Alibaba 分别表示腾讯和阿里数据中心的不同数据集。R 和 F 则表示机器学习中预测手段的评估指标，在章节4.3.1中有详细介绍。表中列举了五种不同的预测模型，RF（Random Forest）[6]、NN（Neural Network）[6]、AE（Auto Encoder）[15]、LSTM（Long Short Term Memory）[38] 以及阿里对随机森林算法的改进版本 MVTRF[12]。可以看到，目前的一些模型算法在 SSD 故障预测领域普遍只能获得约 40% 左右的召回率，这也就意味着仅有约 40% 的故障设备能够被提前预知。这还是在准确率较低的情况下达到的性能，也就是存在着较多的误判代价。如果要减少误判，则会进一步减少预测出来的故障设备数量，这样的性能对于实际部署来说仍旧不足。

表 4.1　各类预测模型在不同数据集下的性能对比

方法	PM1733 Tencent		PM9A3 Tencent		MB1 Alibaba		平均	
	P	R	P	R	P	R	P	R
RF	0.58	0.31	0.75	0.33	0.56	0.37	0.63	0.34
NN	0.63	0.14	0.85	0.31	0.72	0.46	0.73	0.30
AE	0.54	0.14	0.54	0.33	0.36	0.46	0.31	0.35
LSTM	0.36	0.40	0.52	0.25	0.63	0.61	0.50	0.42
MVTRF	0.90	0.40	0.70	0.42	0.89	0.76	0.83	0.53

因此，本文希望基于获取到的真实运维数据，并得出一些规律性结论来提升预测性能，即利用第三章所述的 SSD “状态” 分离潜在的故障原因，进而减少数据集的混乱程度来提升最终的预测性能。然而如何将 “状态” 实际生成以及实际在故障预测中应用，其实还有以下挑战需要讨论和解决：

（1）　作为生成 “状态” 的第一步，如果无法有效的将和故障相关的属性筛选出来，那

中国知网　https://www.cnki.net

么后续的所有操作都会变得没有意义。因此究竟哪些属性需要被选择用来生成状态？通过什么手段来进行属性的选择？这些都是需要着重考虑的问题。

（2） 其次，如何对属性的值范围进行划分。每一个不同的属性其值都有着不同的分布情况，如何进行划分，以及如何确定这样的划分对于潜在故障原因的区分是有帮助的？

（3） 最后就是如何控制“状态”的数量。因为从先前状态的定义来说，似乎会存在指数增加的情况，也就是当属性数量增加，属性区间的数量增加时，“状态”的数量会呈现出爆炸式增长的情况。然而数据集总量是不变的，每一个数据样本又都要对应到一个“状态”中，势必会导致某些“状态”的数据量过小。那么此时，也不应该对其进行单独处理了。对于机器学习算法来说，数据量过少对结果的影响甚至可能大于数据混乱带来的危害。因此如何设置属性数量以及区间数量，或者说如何控制整体“状态”的数量也是一大难题。

4.2 设计：自动化数据预处理架构 Diff-State

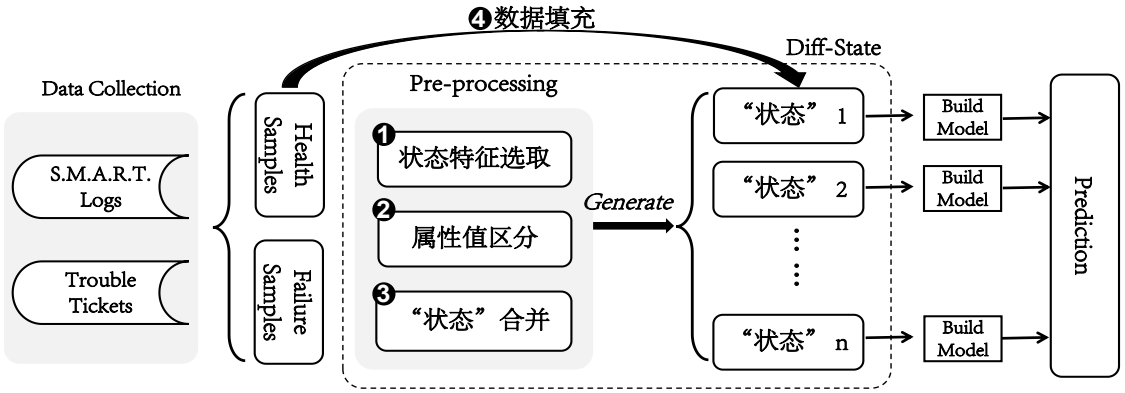


图 4.1 Diff-State 的整体架构

为解决上述挑战，本文提出了一种自动数据预处理方案–Diff-State，以完善“状态”应用并提高故障预测的准确性，如图 4.1所示。由于 Diff-State 是一种数据预处理方案，因此前置的操作和常见流程一样，首先是数据收集，在本文中包含 S.M.A.R.T. 日志信息以及运维人员提供的故障单。同时，通过常规数据预处理，解决掉数据缺失以及属性值类型不统一的问题。

中国知网 https://www.cnki.net

Diff-State 的流程中主要包含 4 个步骤，即状态特征提取（步骤❶）、属性值划分（步骤❷）、”状态” 合并（步骤❸）以及最后的数据填充（步骤❹）。最终的模型建立，使用了随机森林分类法（Random Forest [39]）。接下来将会详细介绍每一个步骤的具体做法和其解决的问题。

4.2.1　状态属性选取

第一个步骤是状态属性选取。特别需要注意的是，这与现有的特征选择工作 [11] 不同。现有的常说的特征选择工作旨在精简模型的规模，优化计算性能，这一步骤通常被安排在数据预处理之后，也就是说在进行状态属性选取的步骤之后依旧会进行常规的特征选择，这二者并不冲突。同时，尽管这二者的方式有些类似，都是寻找和故障相关的属性，但是目的有所不同，状态特征选取的目的是为了生成” 状态”，因此它只会关注最相关的那些属性，而常说特征选择则没有那么严格。简单来说，这是一个新添加的全新步骤，而不是取代原来的步骤。

为了确定状态生成的关键属性，本文选择使用斯皮尔曼相关系数（Spearman’s correlation coefficient [40]）对这些关键属性进行排序。斯皮尔曼相关系数是一种非参数指标，由 Spearman 在 1904 年提出，用于衡量两个变量之间的单调关系。一般来说单调关系指的是两个变量之间联系的强弱，即单调负相关或是单调正相关。如果一个变量增加，那么另一个变量也会增加（单调正相关），或者如果一个变量增加，那么另一个变量会减少（单调负相关）。斯皮尔曼相关系数的取值范围在-1 到 1 之间，越接近 1 或-1 表示两个变量之间的单调关系越强，0 表示两个变量之间没有单调关系。如果斯皮尔曼相关系数的取值为正，则说明两个变量呈单调正相关，如果斯皮尔曼相关系数的取值为负，则说明两个变量呈单调负相关。对于先前的状态定义来说，正相关和负相关同样重要，因此直接取绝对值进行观测。同时，由于数据集比较大，为了避免计算时间过长，本文将在随机抽取健康样本和故障样本后计算相关系数。

最终的结果如表 4.2所示，可以得到每个属性与故障相关性的排序。本文选择前 7 个属性作为关键属性参与状态区分。像先前说的，在状态属性选取这个阶段，要求最相关的属性用来生成“状态”，而从排名第八的属性（Read_write_percent）

中国知网　https://www.cnki.net

表 4.2　故障相关度最高的 7 个属性

属性	Unc	Psf	NAND_type	Gbb
值	0.090	0.079	0.066	0.065
属性	Wa	Dwpd_real	Poh	
值	0.050	0.049	0.034	

开始，其相关系数的绝对值低于 0.010547。与排名前七的属性相比，数值差距较大。因此对这些属性不再做考虑。事实上，从实际得到的斯皮尔曼相关系数排序列表来看，也可以看到数据中心的 SSD 故障预测是一个十分复杂的任务，即使是最相关的 unc，也仅有 0.090 的绝对值相关性。

4.2.2　属性值区分

根据上述”状态”定义，需要将筛选出的关键属性划分为不同的分布范围。为了能够更好的区分开可能存在的分布趋势，因此我们使用贝叶斯变化点检测（Bayesian change point detection [41]）来进行范围分类。贝叶斯变化点检测是一种用于检测序列中突变点、结构变化或者趋势分析的方法。它基于贝叶斯统计方法，通过考虑数据的先验分布和后验分布来确定变化点的位置和数量。本文另辟蹊径，通过对属性值的累积分布图使用贝叶斯变化点检测，来确定值区间趋势的差异。

除了使用哪种方法来划分区间，属性值区分这个步骤也决定了具体应该划分多少个区间，以此来控制最终“状态”的数量。本文将值区间的数量设为 2。这其是一个权衡决定。一方面，许多属性在实际测试中找不到更多有效的变化点。另一方面，这确实也是为了限制最终生成的状态数量。设置统一的值区间数还能有效降低后续处理的复杂性，使部署工作更轻松。事实上，在后续的实验过程中，尝试了不同的数值，但由于存在后续合并等限制，实际上差别并不明显。

设置“状态”的目的是通过不同的“状态”来区分不同的故障表现，从而训练出有针对性的模型，提高后续的故障预测能力。因此，最终用于“状态”设置的属性需要满足两个条件。

- 条件一: 这些属性与故障的相关性很高。

中国知网　https://www.cnki.net

- 条件二: 这些属性的不同值区间可以区分不同的故障表现。

Algorithm 1 条件二验证算法

Input: Failure samples and Healthy samples

1: $Health \leftarrow Random_samples(Health)$
2: $Ranking \leftarrow Spearman(Health + Fail)$
3: **repeat**
4: $A \leftarrow pop(Ranking)$
5: $c_p \leftarrow Bayesian_change_point(Fail[A])$
6: $set1 \leftarrow (Fail + Health)[A\ value <= c_p]$
7: $set2 \leftarrow (Fail + Health)[A\ value > c_p]$
8: $R_s1 \leftarrow Spearman(set1)$
9: $R_s2 \leftarrow Spearman(set2)$
10: $P_value \leftarrow Paired_t\text{-}test(R_s1, R_s2)$
11: **if** $P_value <= 0.2$ **then**
12: $Retention\ attribute\ A\ division$
13: **else**
14: $Discard\ A\ in\ the\ State\ settings$
15: **end if**
16: **until** $Ranking\ is\ empty$

Output: Attributes used for state settings and their change points

先前的斯皮尔曼相关系数能够保证条件一。为了判断这些属性是否满足条件二，本文设置了一个校验算法来进一步验证属性，如算法 1所示。首先，需要对所有的健康样本进行随机采样得到一个较小的健康样本集，即算法中的 $Health$（第 1 行），同时，利用斯皮尔曼相关系数获得健康样本集和故障样本集合集的相关属性排序，即算法中的 $Ranking$（第 2 行）。接下来，具体来说，每次取出相关性排名最高的属性，标记为 A，并找出故障样本中属性 A 分布的变化点 c_p（第 4-5 行）。根据得到的变化点 c_p，将健康样本集和故障样本集合集分为两个子集，即算法中的 $set1$ 和 $set2$，并再次对这两个子集进行斯皮尔曼相关性排序，得到两个新的相关属性序列，即算法中的 R_s1 和 R_s1（第 6-9 行）。此时如果这两个序列中的属性排序出现显著差异时，表明故障在这些属性上的表现发生。因此，继续使

中国知网　https://www.cnki.net

用配对样本 T 检验 [42] 检验，配对样本 T 检验用于检验两个配对样本的数据之间是否存在显著差异。配对样本 T 检验常常出现在生物医学研究中，常见的情形有：配对的受试对象分别接受不同的处理（比如将实验用的小白鼠设置成两个组，分别接受不同的药物，检验处理结果的差异）；或者同一对象的两个部分接受不同的处理（如对于同一批血清样本，平均分为两个部分，对于同一批的两个不同部分进行不同的处理，最后检验结果差异）；以及同一对象的自身前后对照（如检验癌症患者术前、术后的某种指标的差异）。这和分类后的数据样本差异十分类似，因此十分契合。最终，通过配对样本 T 检验能够得到一个 P_value 用来衡量最终的差异大小，这个值越小，意味着差异越大。回到算法中，当得到的 P_value 低于 0.2 时（即置信度为 80%），此时两个序列具有显著性差异，因此保留属性 A 的分割结果。否则，属性 A 的切分结果将被丢弃（第 10-15 行）。对于那些不满足第二个条件的切分，可能是无效切分，最终无法改善预测效果，同时增加了后续流程的复杂性。在这种情况下，需要及早将其删除。

4.2.3 “状态”合并

在前面的步骤中，已经获得了用于设置“状态”的属性以及每个属性的分布情况，因此其实已经可以划分出具体的“状态”。对“状态”进行逐步切分时，实质上就是对数据集进行逐步切分。然而，这就出现了样本稀疏的问题，因为符合某个“状态”的数据样本数量可能太少。在这项工作中，本文将低于总数据样本 5% 的数据样本视为稀疏样本。对于机器学习方法来说，数据样本太少是一个灾难性的问题，会极大地影响机器学习算法的效果。因此，需要有一个合适的算法进行“状态”的合并。

算法 2显示了“状态”合并的过程。首先，需要获取到已经得到的所有“状态”集，即算法中的 $Sets$。紧接着，获得样本量不足的目标“状态”，即算法中的 $State_target$，然后从算法 1获得的最终属性序列中获得排名最低的属性 A 和剩余的属性 $Last$（第 3-5 行）。其次，只需要在“状态”集合中寻找在属性 $Last$ 上与 $State_target$ 范围相同，但在属性 A 上范围不同的“状态”（第 6-12 行）。该”状态”可用于与 $State_target$ 合并。这样，通过合并最不重要属性的范围，就可以

中国知网　https://www.cnki.net

解决样本稀疏的问题，同时尽可能保留先前获得的先验处理。

Algorithm 2 “状态”合并

Input: State data samples, attribute_ranking

$Sets \leftarrow State\ data\ samples$
repeat
  $State_target \leftarrow get_poor_state(sets)$
  $A \leftarrow get_last(attribute_ranking)$
  $Last \leftarrow attribute_ranking - A$
  for $state\ in\ sets$ **do**
    if $state[Last] = State_target[Last]$ **then**
      if $state[A] != State_target[A]$ **then**
        $merge(state, State_target)$
      end if
    end if
  end for
until $There\ is\ no\ poor\ state$

Output: Final state data samples

4.2.4　数据填充

在对数据进行预处理以获得“状态”之后，下一步就是使用故障样本和健康样本进行建模训练。请注意，上述步骤是在故障样本上进行的，因为本文关注的是故障性能的差异。为了更好地训练模型，必须让健康样本适合“状态”。健康样本的处理过程要比故障样本的处理过程简单得多，因为先前已经给出了用于区分的属性和属性阈值。根据这些规则，就可以很简单的对健康样本进行分类。这样，就为后续的每次训练提供了不同“状态”的完整数据集。

4.3　实验与分析

在本节中，将会首先介绍数据集的设置和评估中使用的指标，然后通过使用华为数据集进行跟踪实验，评估 Diff-State 在固态硬盘故障预测准确性和系统性能方面的有效性。

中国知网　https://www.cnki.net

4.3.1 实验设置

数据集设置

本文从华为数据中心收集了用于故障预测的数据集。该数据集的时间跨度为 2017 年 10 月至 2021 年 9 月。为避免模型过拟合问题，将数据集分为三部分：训练集、验证集和测试集。训练集用于模型学习。验证集和测试集用于调整模型的超参数和初步评估模型的性能。它们的初始比例分别设置为 6:2:2。所有实验都使用了 H2O 自动机器学习框架 [43]。H2O 是一个开源、分布式、基于内存、可扩展的机器学习和预测分析框架，用于在数据中心构建大规模机器学习模型。由于机器学习算法的随机性，实验结果为 10 次实验后的平均值。

评估指标

由于硬盘故障预测是一个二元分类问题（即故障或健康）。因此重点关注以下指标，这些指标已在之前的工作中介绍过 [44][15]。

- **True Positive Rate (TPR)**: 预测为故障的设备数量与实际故障设备总数的比率。
- **False Positive Rate (FPR)**: 错误预测的故障设备（实际健康）数量与健康设备总数的比率。
- **Accuracy Rate (ACC)**: 模型正确判断的数据与全部数据的比率。
- **Precision Rate (PRE)**: 正确预测故障设备数量与预测故障设备数量的比率。
- **Recall Rate (RECALL)**: 预测为故障的设备数量与实际故障设备总数的比率。

实验内容

为了评估 Diff-State 的优势，本文首先构建了实验来收集预测性能，然后分析算法 1 和算法 2 的有效性，同时，还对模型学习运行时间进行了评估。主要实验内容如下：

- Diff-State 的故障预测性能。(Exp #1)
- 算法 1 的有效性。(Exp #2)
- Diff-State 对模型学习运行时间的影响。(Exp #3)

中国知网 https://www.cnki.net

4.3.2 Diff-State 的故障预测性能评估

本文比较了 Diff-State 与三种预测模型的预测准确性，包括随机森林（Random Forest，简称 RF）([39])、XGBoost（XGB）([45])、梯度提升机（Gradient Boosting Machine，简称 GBM)([46])以及最先进的故障预测方法多视角(MUL-VIEW)([12])。对于每种故障预测方法，本文都会遍历 0-20 天的回溯周期，以找到预测精度最高的方法。此处的回顾其其实指的就是预测周期。因为对于预测来说，需要有一个时间提前量，一般来说这个提前量会被设置为 7 天。也就是说预测固态硬盘是否会在未来 7 天内发生故障。为了达到这个目的，就需要设置回溯周期，具体的实现方式是在样本选择过程中决定的。以 7 天为例，选择样本时，将故障当天以及故障当天前 7 天的样本设置为故障样本。这样，只需要在分类时待预测设备样本能够被模型分类为故障，就意味着当前待预测设备样本的状态和先前故障设备故障当天或者故障前 7 天的状态类似，也就可以认为待预测设备样本未来 7 天内会发生故障。对于每个模型，经过筛选后，本文使用效果更好的参数设置。同时，Multi-view 使用了其论文中声称的较好结果进行比较。一些主要参数设置如下：RF（$ntrees$=100，max_depth=50），XGB（$ntrees$=100，max_depth=50），GBM（$ntrees$=50，max_depth=20）。

图 4.2显示了这些故障预测方法的性能差异，本文首先从三个指标对它们进行比较：ACC、PRE 和 RECALL。从结果可以看出，虽然 MUL-VIEW 对数据源进行了处理，从不同角度收集数据，从而更全面地覆盖故障表现。但是，由于数据集的不同，结果看起来并没有本文所做的其他实验那么高。相比之下，基于决策树的算法更加稳定。可以看到，在使用 Diff-State 之后，RF、XGB 和 GBM 的预测性能都有了一定的提升。本文建议的 Diff-State(RF) 可以将 PRE 和 RECALL 保持在较高水平，当 RECALL 达到 0.91 时，PRE 可以保持在 0.81。值得注意的是，虽然在本处没有用图表进行展示，但是本文还对另外一个最新工作进行了测试 [15]，并且该测试运行在本文的数据集上。本文测试了先前的一分类算法性能，即孤立森林（Isolation Forests，简称 IF），使用参数 ($ntrees$=100)。但是 IF 的效果并不好，在最后的结果中，当 IF 的 RECALL 值达到其他算法水平时，有近四分之一的健康设备被预测为故障，导致 ACC 和 PRE 的性能较差。这反映出像 IF 这样的单分类模

中国知网 https://www.cnki.net

型并不能很好地用于数据中心数据集的故障预测。这是因为类似的离群点算法需要对数据集进行高度区分。一旦健康样本和故障样本在某个属性上的值相互交叉，就会影响单一分类模型对边界的判断。而类似决策树这种有监督模型则在单属性上的交叉影响较小。

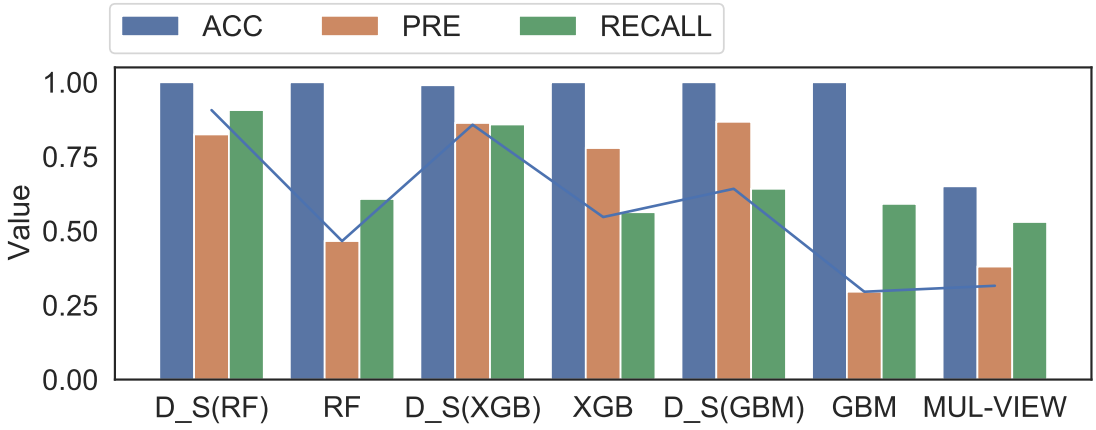


图 4.2 各类方法使用 Diff-State 或者不使用时的预测性能比较，折线代表各种方法在 FPR 为 0.1% 时的 TPR 值

其次，将 FPR 控制在 0.1% 后，本文比较了每种方法的 TPR。如图 4.2中的折线所示，可以看到 Diff-State 仍然能够保持相对稳定的水平，并且能够将 TPR 控制在 0.9 以上，拉开了与其他方法的较大差距。即使是第二高的 XGB 也只能达到 0.54，仍有 0.3 以上的差距。

这是由于 Diff-State 对故障的分类具有启发性。为了进一步验证区分不同“状态”设备下的预测性能，本文从最具代表性的通电时长入手，从属性名称、相关性、标准化相关性和百分比的角度，列出了新旧设备在训练模型中最相关的 6 个 S.M.A.R.T. 属性（见表 4.3）。该表展示的是随机森林建模中，最终模型自身对哪些属性的依赖性最高。可以明显看到年轻设备和年老设备的结果有显著差异。对于新设备，Fbb 相关性相当高，这意味着部署后不久出现的故障很可能是由于固态硬盘本身质量不佳造成的。对于旧设备，我们发现 Life_expect 和 Lpu 这两个属性的相关性非常高。Life_expect 表示剩余 P/E 周期的数量，Lpu 表示已使用 P/E 周期占供应商指定 P/E 周期的百分比。这表明，闪存磨损是导致固态硬盘部署时间较长后

中国知网 https://www.cnki.net

才发生故障的主要因素。总之，基于“状态”的划分使故障预测模型更容易处理相对统一的数据，从而提高了预测性能。

表 4.3　新旧设备最相关的 6 个 S.M.A.R.T. 属性

(a) 旧设备故障相关属性			
属性	**故障相关度**	**归一化相关度**	**%**
Life_expect	134.485367	1.000000	0.114413
Slot	124.912109	0.928816	0.106268
Lpu	106.148781	0.789296	0.090305
Host_write_lba	100.974754	0.750823	0.085904
Temp_highest	85.856873	0.638411	0.073042
Before_raid_unc	82.489326	0.613370	0.070177
(b) 新设备故障相关属性			
属性	**故障相关度**	**归一化相关度**	**%**
Slot	110.635063	1.000000	0.067235
Fbb	91.326759	0.825478	0.055501
Read_write_percent	84.231033	0.761341	0.051189
Poh	77.175247	0.697566	0.046901
Pot_month	64.389450	0.581999	0.039131
Frame	61.674835	0.557462	0.037481

4.3.3　Diff-State“状态”区分有效性的验证

如先前介绍，算法1筛选并评估哪些 S.M.A.R.T. 属性确实符合” 状态” 分类的条件 2。如表 4.2所示，本文使用数据集可以筛选出 7 个最相关的属性。接下来展示经过算法1之后得到的结果，如表4.4所示。为了避免无用裁剪，本文将置信度设置为 80%，即 P_value 小于 0.2。因此，NAND_type、Dwpd_real 和 Poh 符合要求。为了进行比较，我们直接选择同一数据集中与” 状态” 分类相关性最高的三个属性（即 Unc、Psf 和 NAND_type ）。

图 4.3显示了 RF、W/o Alg1 和 Diff-State(RF) 三种评估方法的预测性能。其中，RF 表示不经任何处理的随机森林。W/o Alg1 代表未使用算法 1 的 Diff-State。从结

中国知网　https://www.cnki.net

表 4.4 算法 1后得到的 7 个最相关属性的 P_value

属性	P_value
Unc	0.816153
Psf	0.779035
NAND_type	0.166501
Gbb	0.673579
Wa	0.682550
Dwpd_real	0.042712
Poh	0.062030

果来看，W/o Alg1 的预测性能甚至不如 RF，这说明只选择最相关的属性会导致许多无用的切分。也就是说，无用切分越多，会导致样本量越小，反而会降低模型的预测性能。Diff-State(RF) 能提高模型的预测性能。这是因为 Diff-State 对” 状态” 进行了划分，可以区分故障的潜在差异。

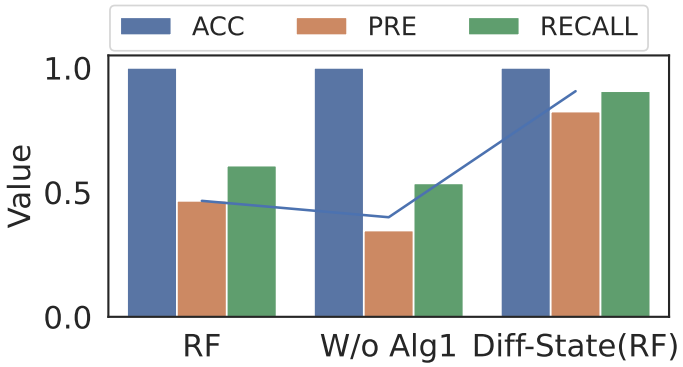


图 4.3 使用算法1及不使用时的预测性能比较, 折线代表各种方法的 TPR 值

4.3.4 Diff-State 模型学习时间评估

本文使用 H2O 云服务器上的数据集来评估每个预测模型的运行性能。对于随机森林（Random Forest）、XGBoost、梯度提升机（Gradient Boosting Machine）和孤立森林（Isolated Forest），我们使用了 100 棵树，并将最大深度设置为 30。表 4.5显示了预测模型和 Diff-State 的平均运行时间。运行时间只包括模型构建时间，

中国知网 https://www.cnki.net

不包括样本预处理时间。这是因为 Diff-State 的处理时间很短，因为主要处理的是故障样本，而故障样本只占所有样本的一小部分。从结果中可以发现，建立多个 Diff-State 模型并不会增加整体运行时间。事实上，通过“状态”的划分，将数据进行分割之后，数据集规模被分割，有利于模型的搭建。同时，需要注意的时，本表中展示的时间依旧是串行方式进行，也就是执行完上一个“状态”的数据集之后再执行下一个。但是实际上，不同“状态”下的数据集之间并没有联系，可以通过并行的方式进一步加速模型搭建。

表 4.5　模型训练时间

模型	Diff_State(RF)	RF	Diff_State(XGB)	XGB
时间 /秒	130	183	601	1398
模型	Diff_State(GBM)	GBM	IF	
时间 /秒	129	377	76	

4.4　本章小结

本章提出了 Diff-State，一种自动数据预处理架构，以完善“状态”应用并提高故障预测的准确性。通过四个步骤即状态特征提取、属性值划分、”状态”合并以及最后的数据填充，能够将数据集切分为多个子数据集，并贴合不同的“状态”。再通过对不同的子数据集进行单独的建模，最终得到了不错的效果提升。在随机森林算法中，使用了 Diff-State 之后，当 RECALL 达到 0.91 时，PRE 可以保持在 0.81，相比较没有使用 Diff-State，PRE 可以提升约 0.4，RECALL 可以提升约 0.35。同时，用实验证明了算法 2 的有效性，能够确保“状态”区分的正确性。最后，用实验证明了 Diff-State 并不会增加整体运行时间，相反，由于分割之后，数据集规模被分割，有利于模型的搭建，整体运行时间反而缩减。

中国知网　https://www.cnki.net

中国知网 https://www.cnki.net

第五章　采样优化：预测模型的回溯周期优化

在本章中，针对先前提到的基于机器学习的 SSD 故障预测方案，在样本选择环节进行了进一步优化。值得注意的是，优化的内容并不是提升模型的性能，而是让模型能够更好的实际部署。

5.1　研究动机：训练样本选择不准确

5.1.1　常见样本选择方式

先前已经提到过，在进行数据收集时，对于故障样本的选择会有一个选择的概念。对于故障样本的选择是一个非常重要的步骤，因为对于决策树这类有监督机器学习算法来说，设置的故障样本即为其学习的对象。所有算法认为和这些故障样本接近或者相似的都会被预测为故障。一般来说，目前的样本有两种。接下来进行详细介绍。

Baseline。这是最基本、最简单的划分方法。如图 5.1的（a）所示，这种方法只选取故障当天的 SSD 样本作为正样本，而将其他生活中的样本视为健康的负样本，也就是说模型会直接学习故障当天的属性数据模式。从模型的角度来看，这样划分训练出来的模型对故障模式的识别能力最为灵敏。然而，这也存在一些问题。一方面，在故障硬盘和健康硬盘数量已经严重不成比例的情况下，只选择故障当天的样本作为正样本，意味着将原来的比例差异进一步加深几十倍甚至几百倍。同时，虽然这种方法可以让模型直接学习故障模式，但却忽略了” 预测” 的初衷，变成了单纯的故障设备识别，即模型的最终” 目标” 变成了故障的固态硬盘，这使得最终结果在实际部署中毫无意义。

回溯周期。这种方法是设置一个追溯期，如图 5.1的 (b) 所示。将 T_0 定义为故障发生的那一天，将 T_n 定义为故障发生前的第 n 天。当回溯周期的长度被设为 n 时，这意味着我们将目标设为包含在 T_1~T_n 中的 n 天样本，并将这 n 天的样本设为正样本。之所以可以这样设置，是因为目前的研究普遍认为，相应的特征会在 SSD 故障发生前几天出现，这些特征也会在 S.M.A.R.T. 等属性值中表现出来。与

中国知网　https://www.cnki.net

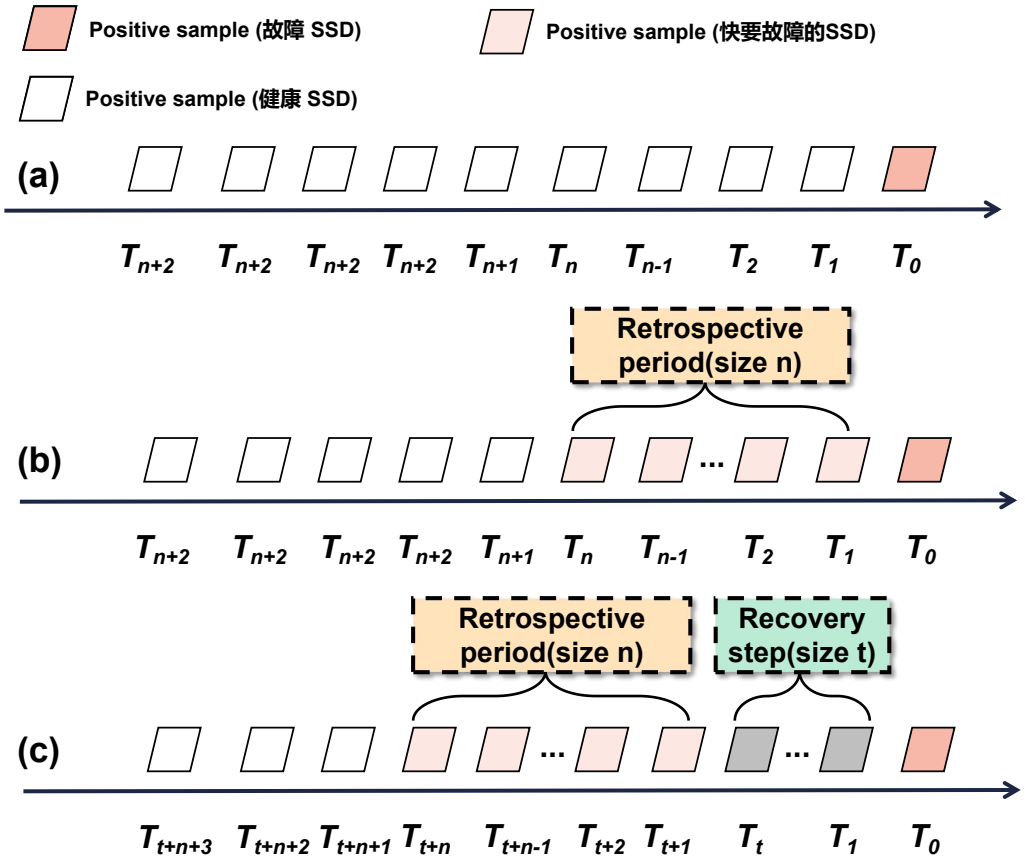


图 5.1　三种不同的回溯周期

故障样本接近的样本也会被用作正样本，让模型学习故障模式。就结果而言，这意味着模型预测的最终阳性样本代表了整个设备在 1 到 n 天后发生故障的可能性。

5.1.2　样本选择问题分析

以上两种方式对于未出现故障的健康设备均没有进行处理，即将这些设备的所有样本都标记为健康样本。然而，尽管回溯周期解决了 Baseline 中单纯分类的问题，但是这样的样本选择依旧存在其他问题。

首先，对于故障样本来说。进行 SSD 故障预测的初衷，是为了保护设备中存放的数据，进而保障整体系统服务的稳定运行。然而，在这样的回溯周期情况下，依旧会出现这样的情况：假设如图 5.1的 (b) 所示，T_1~T_n 中的 n 天样本被标记为故障样本。此时算法成功将一个即将故障的设备预测出来，但是实际上其对应的相似样本为 T_1。也就是从逻辑上来说，该设备明天就会发生故障。此时的预测

中国知网　https://www.cnki.net

手段其实已经没有作用了，因为对于数据的保护同样需要读取操作，那么对于这个设备来说，很有可能在数据保护的过程中出现故障。由于设备通常集群工作，负载会分散在不同的设备中，甚至可能出现这个设备其实已经故障，只是没有被访问而没被发现而已。因此，对于故障样本来说，预测算法需要从逻辑上为数据的保护进程留出足够可用时间。

其次，对于未出现故障的健康设备来说，将其的所有样本都标记为健康样本并不是一个严谨的操作。因为故障其实是一个后验的情况，在这个设备还健康时无法确定未来是否会发生故障。那么对于这些当前还未出现故障的健康设备来说，很可能存在这样的情况，即在收集到当天数据的后一天就会发生故障。这是难以避免的，并且当这样的情况发生，对于预测有着极大的打击，这是因为我们会将实际应该标记为故障的样本错误标记为健康。这会极大干扰模型的判断。

为了验证之前提到的问题，本文统计了所有故障设备在故障前 1-9 天的工作负载。同时还统计了在不同回溯周期下，写入负载与故障当天收集的写入负载相同或仅相差 100MB 的情况所占百分比。

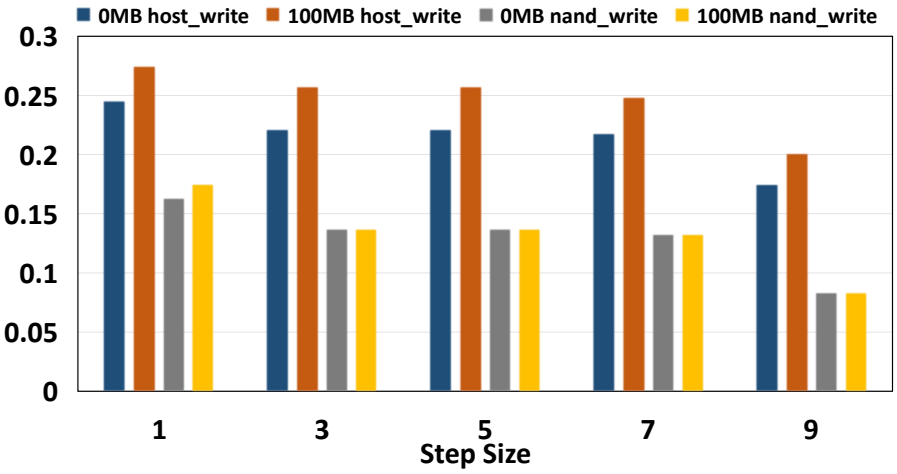


图 5.2　故障发生当日附近的负载状况。纵坐标以百分比为单位，横坐标为时间步长，即故障发生前的天数。

如图 5.2所示，在故障发生前一天，工作负载为 0MB 或小于 100MB 的情况占了不小的比例。请注意，这里的数据量指的是故障前后的差异。主机写入的指标约为 25%，而 nand 写入的指标约为 16%。随着时间的推移，工作负载的差异越来越

中国知网　https://www.cnki.net

大。值得注意的是，这里的主机写入量是经过上层缓存后的主机写入量，由于一些垃圾收集操作，上层缓存产生的写入量要少于 nand 写入量。

这说明故障发现确实可能存在延迟。甚至有接近四分之一的故障设备在发现其为故障的前一天没有任何工作负载。也许设备很早以前就发生了故障，或者已经达到了故障临界状态，但由于没有产生工作负载而无法被发现。这可能会给预测带来问题，因为我们认为的故障前临界状态很可能已经是故障状态。有理由相信，这将导致预测性能膨胀，因为预测故障前状态比对其进行类似 Baseline 的分类更加困难。

5.2　设计：带有恢复步长的样本选择优化

为了解决上述的问题，在选择健康样本和失败样本时，本文都设置了一定长度的恢复步长。对于健康样本，恢复步长可以确保标记的正确性。避免一些即将失效的设备，但仍然会将其标记为健康，而对于哪些本来就是健康的设备，无非是少几天健康样本的数据，这对于整体的故障预测来说影响几乎为零。对于故障样本，恢复步长可以确保预测的故障设备与真正的故障时刻有一定的时间间隔，提高数据的可靠性。因此，如图 5.1中的 (c) 所示，将在回溯期之前再设置一个恢复步骤。当我们将回溯周期的长度设为 n，恢复步骤设为 t 时，意味着我们将目标设为 T_{t+1} ~ T_{t+n} 中包含的 n 天样本。

尽管这样的改动看上去很简单，但是却能够很好的解决之前提到的问题。如图5.3，展示了选取该样本为健康样本和故障样本时的情况。

5.2.1　故障样本优化情况

T_0 为 S.M.A.R.T. 当前收集到的数据所展示的情况，因为当前 SSD 已经发生故障，因此该设备选择的 n 天数据样本会被标记为故障样本，让模型进行学习，所有和这 n 天内数据情况类似的 SSD 都会被标记为未来 n 天内可能会发生故障。然而正如上一子节统计的一样，有相当一部分设备在故障前并没有工作负载，因此可能该设备早已发生故障但是却没有被发现，即可能在选择的 n 天数据中，就已经是发生故障的数据情况。这让预测本身转变成了判断，模型只能区分已经发生故障

中国知网　https://www.cnki.net

的设备和还没发生故障的设备，失去了预测即将发生故障设备的能力。但是，当设置了恢复步长之后，就可以有效避免该问题，由于恢复步长的这段数据会被舍弃，因此这一类潜藏的故障设备出现在选择的样本中的概率就会大大减少。

5.2.2　健康样本优化情况

由于 T_0 当天设备依旧正常运行，因此该 SSD 会被标记为健康设备。因此该设备选择的 n 天数据样本会被标记为健康样本。然而，可能存在这样的情况，即 S.M.A.R.T. 当前收集到的数据之后，这个设备就会出现故障。此时，对于选择的 n 天样本来说，实际上可以称之为即将发生故障的数据样本，而这真是最佳的故障样本，但是在这种情况下却被标记为健康样本。这对模型的伤害很大，因为这会混淆模型对健康和故障两种状态的判断。但是加入了恢复步长之后，就可以去掉这些故障样本，由于选择的样本距离当前更远，因此将其标记为健康样本并不会有问题。

5.2.3　恢复步长对预测性能的影响分析

正如前面介绍的一样，对于回溯周期的优化目的是为了减少样本选择时的错误，避免错误的样本对预测模型的判断产生影响进而影响最终的预测性能。然而，这部分优化的性能其实并不像第章节一样能够通过实验的方式展示出来，甚至在实验中可能出现性能下滑的问题。但是这并不意味着这样的改动是无用改动，造成这样的原因主要有两个，接下来的章节将进行详细说明。

影响原因一：健康样本未来情况不确定

如图5.3所示，对于健康样本的优化是考虑到了未来可能发生的情况。即使某一个健康的设备的确会在未来几天内很快故障，在训练模型时也无法将其视为故障设备，因为当前获得的数据无法得知其未来是否会发生故障，这本身就成了预测的目标。然而这样的情况确实无法被忽视，尤其是本文已经通过数据统计得出，有接近四分之一的故障设备在故障前数天内并没有负载访问。因此，或许当前设备被标记为健康仅仅只是因为没有负载访问。四分之一并不是一个小份额，尤其是对于需要学习故障样本本身特性的模型来说。

中国知网　https://www.cnki.net

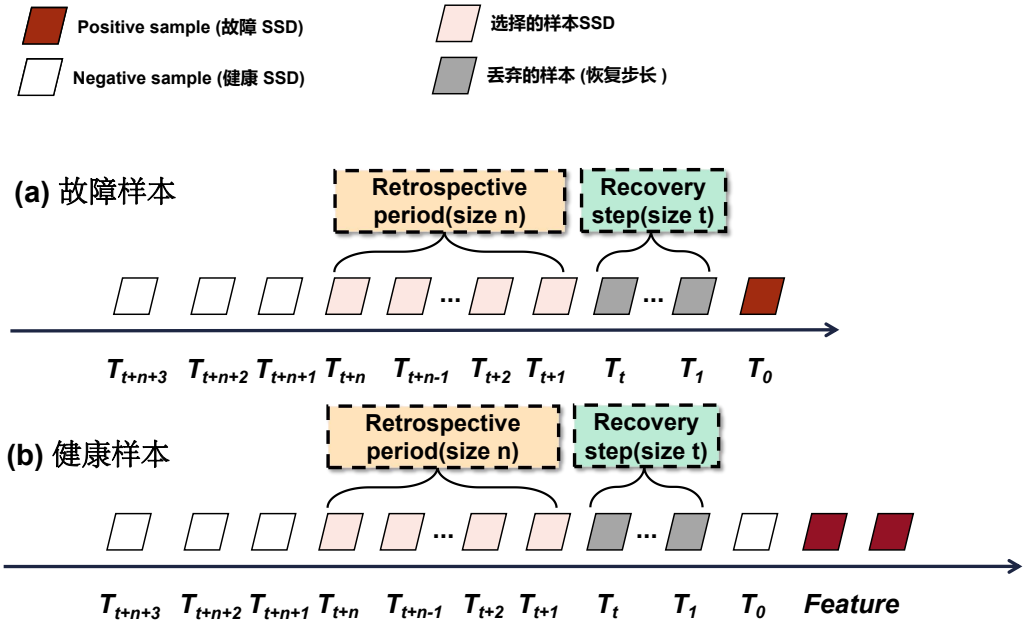


图 5.3　标记故障样本和健康样本的情况

但是从测试角度来说，因为无法将错误的健康样本标回为故障，因此使用了恢复步长之后，对于实验测试的影响仅仅只是相当于删掉了一些健康样本，但是在 SSD 故障预测中，健康样本远远多于故障样本，因此对于最终的性能效果几乎不会有任何影响。

影响原因二：带有问题的样本选择性能虚高

先前提到过，有接近四分之一的故障设备在故障前数天内并没有负载访问，因此在那些被标记为故障的设备上采集到的样本很可能完全相同，仅仅存在使用时间上的差异。同时，这些样本本身其实就是故障时的表现，而失去了代表故障前几天特征的能力。因此，此时的故障预测就会被降级为故障分类，分类对于模型的挑战要小得多，因为这意味着模型不需要去判断一些处于边界的模糊样本。在这种情况下，预测的性能往往会比实际部署后的效果更好。

然而从测试的角度来看，虽然采用恢复步长可以避免这样的问题，但是去掉一些离设备故障当天更接近的样本，确确实实会对最终的预测性能产生负面影响，尽管它本应如此。

中国知网　https://www.cnki.net

5.3　实验与测试

在本节中，将通过实验探索先前提到的回溯周期及恢复步长对于性能的影响以及最佳设置。

5.3.1　实验配置

数据集设置

整体实验环境与先前类似，使用了 H2O 自动机器学习框架。训练集、验证集和测试集的初始比例分别设置为 6:2:2。实验结果为 10 次实验后的平均值。使用的机器学习算法为随机森林（$ntrees$=100，max_depth=50）。

评价指标

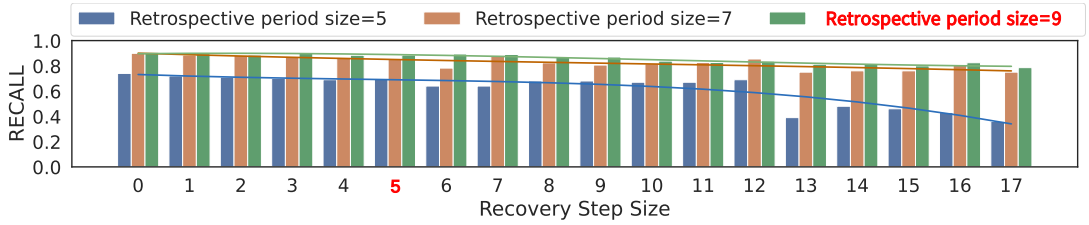
- **Precision Rate (PRE)**: 正确预测故障设备数量与预测故障设备数量的比率。
- **Recall Rate (RECALL)**: 预测为故障的设备数量与实际故障设备总数的比率。

5.3.2　恢复步长对预测性能的测试评估

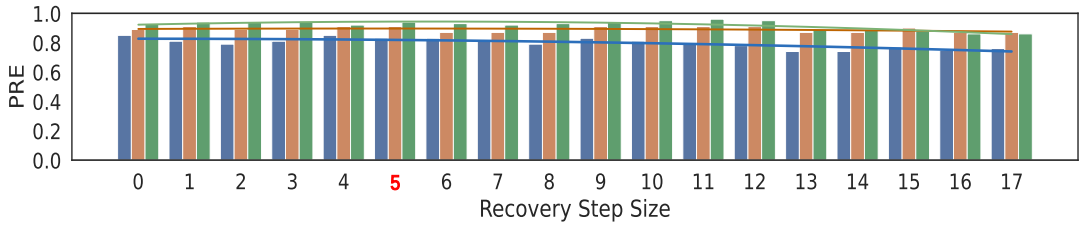
回想一下之前提到的，设置带有恢复步长的回溯周期会对最终的测试性能产生一定的影响，接下来将通过实验展示不同的恢复步长对于最终的预测性能会产生怎么样的影响。同时，通过实验结果对于究竟应该选择多大的恢复步长做出指导意见。

根据以往的工作经验和工程师合作伙伴的知识，固态硬盘的回溯周期约为 5-7 天。因此，我们保守地将追溯周期设置为 5、7 和 9 天，以揭示一些规律。图 5.4显示了通过改变追溯期大小和恢复步长，不同参数设置下的性能趋势。图 5.4(a) 指的是 RECALL 指标，而图 5.4(b) 指的是 PRE 指标。首先，随着追溯期大小开始增大，RECALL 和 PRE 指标的性能会得到改善。当追溯期设置为 9 时，模型性能似乎达到了顶峰。当追溯期大于 16 时，性能开始下降。这是因为随着回溯周期的增大，这些样本可能不足以显示故障样本，从而使模型难以区分这些” 假故障样本” 和真正的健康样本。其次，随着恢复步长的增加，我们可以看到一个明显的下降趋势，尤其是当恢复步长设置为 5 时，RECALL 指标分别下降了约 26% 和 23%。这也可能

中国知网　https://www.cnki.net



(a) Recall Rate



(b) Precision Rate

图 5.4　不同回溯周期大小和恢复步长下的预测性能, 曲线代表性能趋势

是由于” 假失败样本” 造成的。根据上述讨论，回溯周期大小设为 9，恢复步长设为 5。

虽然这种设置看起来会降低预测性能。但需要强调的是，这种性能下降是必要的。由于样本数据的未来存在不确定性，因此下降的性能更接近真实情况。

5.4　本章小结

本章针对现有工作中对于样本选择的问题进行了优化，提出了带有恢复步长的回溯周期，很好的解决了健康样本不准确以及故障样本不及时的两个问题。并通过实验展示了在不同的恢复步长以及回溯周期下，模型性能的变化情况。尽管设置恢复步长之后会带来一定的性能损失，但是这使得最终的性能更加接近真实情况。

中国知网　https://www.cnki.net

第六章　部署优化：基于预测备份的 RAID 快速恢复技术

尽管本文提出的 Diff-State 已经极大的提升了预测模型的性能。但是在实际部署场景，即需要较高 PRE 的情况下，能够预测出的故障盘数量依旧会下降。同时由于 RAID 这类冗余架构在数据中心中经常被使用，因此结合 RAID 和预测的特性，本文创造性的提出了 Precovery，不仅很好的解决了实际部署中预测模型性能较差的情况，也帮助 RAID 能够快速恢复，避免长时间的降级情况。

6.1　研究动机：预测部署受限无法发挥最大性能

6.1.1　动机一：预测部署受限

正如前面介绍，通常来说会有两个重要指标来衡量预测手段的有效性，即 Precision Rate 和 Recall Rate。然而，对于训练有素的模型来说，这两个指标之间通常存在着折衷，即提高一个指标通常会降低另一个指标。为了达到高 Precision Rate，难免会将实际故障设备标记为健康；而为了达到高 Recall Rate 则会将健康设备标记为故障。因此，在预测邻域会选择一个较为平均的均衡点来实现较好的性能。

然而，在数据中心的场景下，部署的需求会有所不同。在数据中心场景下的 SSD 往往属于企业级 SSD，这一类 SSD 与市面上消费级 SSD 最大的差异就是容量更大且价格更加昂贵。最新的企业级 SSD 的容量已经可以高达几十 TB，同时价格也高达数千甚至更高。因此，数据中心中的 SSD 其实是一种十分昂贵的资源，过多的将健康设备标记为故障会产生大量的容量以及经济上的浪费，在这种情况下，高 Precision Rate 更加适合数据中心场景，但是与之对应的，预测出故障设备的能力就会下降。因此，需要有一种机制能够在高 Recall Rate 情况下，进一步保护或是识别那些被误判为故障的健康设备。

6.1.2　动机二：数据中心常见场景 RAID 数据恢复速度慢

虽然 RAID 架构能够保证一定数量的存储设备发生故障时数据不会丢失。但是 RAID 架构会在恢复数据的过程中处于性能降级模式。这是因为按照先前提到

中国知网　https://www.cnki.net

的数据恢复原理，在故障设备上获取丢失的数据块需要额外的读取操作和计算过程，因此总读取性能不可避免地会下降。

表 6.1 RAID5 降级的性能表现

	正常情况下的 RAID5	性能降级情况下的 RAID5
随机读带宽	5347 MB/s	1142 MB/s
顺序读带宽	10600 MB/s	2563 MB/s

为了说明 RAID 的降级问题，本文在一台配备两个 20 核英特尔 XEON CPU 和 384GB DRAM 的服务器上进行了实验，该服务器运行 Ubuntu 22.04 和 Linux 内核 v6.2.0。同时使用四块 1TB ES3500P SSD 作为高密度 SSD，其中三块用于构建阵列，一块用于替换。为简单起见，只在每个固态硬盘上使用 128GB 分区来构建基于奇偶校验的 RAID5。在 RAID 管理系统方面，本文采用了广泛使用的 Linux mdraid [47]，并将 RAID5 作为四个固态硬盘的 RAID 格式。RAID5 的块大小默认设置为 512KB，RAID5 工作线程数设置为 16。对于前台工作负载，使用 FIO [48] 来生成读写工作负载。随机工作负载和顺序工作负载的 I/O 大小分别设置为 4KB 和 1MB。如表6.1所示，与正常的 RAID5 相比，降级后的 RAID5 只能提供 21.8% 的随机读取吞吐量和 24.17% 的顺序读取吞吐量。除了性能下降，容错能力也会下降，因为阵列中的冗余度降低了。对于已降级的 RAID5，另一个存储设备故障将导致阵列中的数据丢失，而且很难恢复。因此，基于奇偶校验的 RAID 必须尽快退出降级模式, 如何让 RAID 能够快速恢复同样是一个极具价值的挑战。

6.2 设计：两阶段的故障预恢复架构 Precovery

为了充分利用预测来帮助缩短降级周期，本文提出了一种两阶段的故障预恢复方法，提前为预测出来的故障设备做备份。大致的原理如下：第一阶段是镜像生成阶段，为预测为故障的设备生成一个镜像设备。该镜像设备可作为原始设备的备份，随时替换原始设备，避免原始设备出现故障。第二阶段是镜像维护阶段，保持原设备和镜像设备的正常运行，等待故障发生。通过镜像维护，预测为故障的设

中国知网 https://www.cnki.net

备不会被立即丢弃，从而使得该机制能够看到设备是否真的发生故障。有了这样的预恢复方式，使得高召回率模型得以实际使用，而不会让误判造成大量的经济和容量损失。同时，预恢复的方式也能够避免 RAID 恢复数据过程中的长时间降级时间。

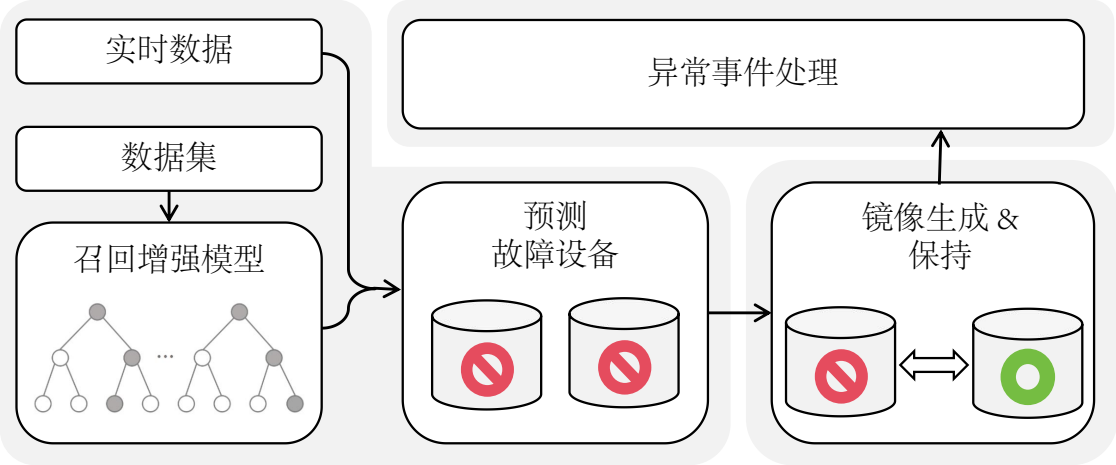


图 6.1　两阶段的故障预恢复架构示意图

图 6.1展示了两阶段的故障预恢复机制的整个流程。首先，模型挑战，即为预测工作离线训练一个召回增强模型。其次，定期获取 S.M.A.R.T. 信息作为模型的数据输入。然后，模型利用累积的 S.M.A.R.T. 信息来预测哪些设备将出现故障。第三，设置预测失效设备的镜像，对于这些预测到的故障设备，进行第一阶段，即镜像生成。生成镜像设备时将使用备用设备（即原始设备的数据将同步到备用设备）。最后，原始设备将被监控一段时间作为观察期。在观察期内，镜像设备将与原始设备同步更新（RAID1 方式）。原始设备发生故障后，镜像设备可立即取代原始设备。如果原始设备在观察期内存活，镜像设备将被释放，并可用于镜像其他设备。

6.2.1　模型调整

如前所述，预恢复的架构依赖于模型的召回率增强版本。因此，如何调整模型的性能是实现该设计的基础。从逻辑上来说，故障预测可视为二元分类任务，模型只需输出 1 或 0 两类。在预测的场景中，1 代表预测故障，0 代表预测健康。在实际操作中，机器学习模型中可以输出待识别样本被识别为 1 的概率。这意味着每

中国知网　https://www.cnki.net

个样本的概率值将分布在 0 和 1 之间。通常情况下，实际有故障的设备的概率值会更接近 1，而实际健康的设备的概率值会更接近 0。然而，总会出现这两类设备的分布相交的情况，这也是预测不可能完美的原因。在这种情况下，给出最终预测结果的方式就是设置一个阈值。如果预测概率高于这个临界值，我们就认为这个样本被预测为会发生故障。反之，则认为预测设备是健康的。通过调整阈值，可以很容易地调整模型，将其设置为高召回率模型（如果需要，也可以设置为高精度模型）。如先前章节 2.2.2中的图 2.3所示，以三个阈值为例进行说明。阈值 3 可以识别所有故障设备，并将一些健康设备也识别为故障设备，此时模型就变成了典型的高召回率模型。同样，阈值 1 代表高精度模型，阈值 2 代表平衡模型。

需要强调的是，在本文的预恢复设计中，并不限制使用什么模型，因此本环节只关注模型的调整。事实上，通过阈值来调整模型性能是一种非常普遍的方法，几乎适用于所有用于分类任务的机器学习算法。在后续的设计中，本文以目前广泛使用的随机森林（Random Forest [39]）为例，用真实的数据建立了一个真实的模型，来验证本设计的合理性。

6.2.2 预测失效设备的镜像生成

为了保护故障预测中的被预测为故障的存储设备，预恢复机制会生成一个镜像设备。镜像设备具有与原始设备相同的内容，可以立即替代原始设备。这样，一旦预测的设备出现故障，镜像设备就能迅速替代故障设备。然而，这种直接的策略需要较高的预测精度，以避免替换错误预测的健康设备，从而增加维护成本。这就限制了预测过程所能发现的潜在故障设备的数量。

为了解决这个问题，本文的预恢复机制还在镜像设备建立后的观察期内维护镜像设备。观察期结束后，如果原始设备仍能正常工作，相关的镜像设备就可以被释放。真正的会发生故障的设备很可能会在观测期内发生故障，而镜像设备可以立即替换故障设备，几乎不会耽误降级时间。镜像设备的维护需要与原始设备同步更新，以保证一致性。因此，本文的预恢复机制将原始设备和备用设备组成 RAID1 阵列，使备用设备成为原始设备的镜像。

图 6.2以 4 个设备的 RAID5 为例展示了这一过程。在此将预测为故障的设备

中国知网 https://www.cnki.net

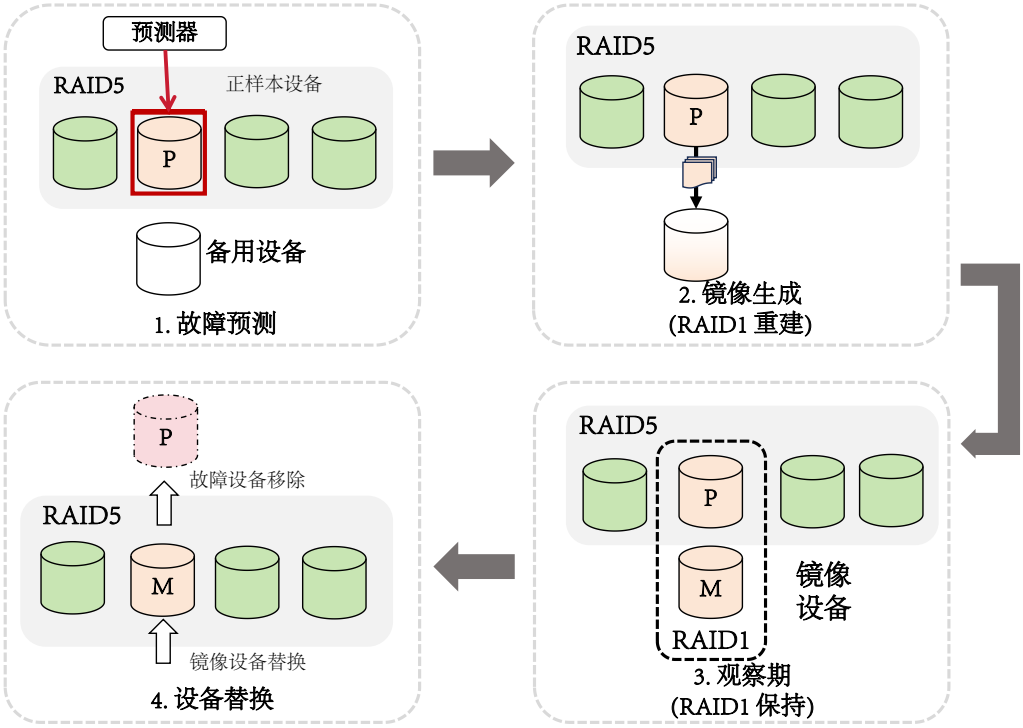


图 6.2　镜像设备的生命周期

称为正样本设备。首先，预测算法会给出一个正样本设备。其次，预恢复机制开始使用备用设备与正样本设备组成 RAID1 阵列，而正样本设备目前仍在运行。第三，在观察期间，RAID1 在上层 RAID5 中作为单个设备运行。RAID1 携带正样本设备上的数据，等待设备故障。最后，在正样本设备发生故障后，移除正样本设备，保留镜像设备。

6.2.3　异常事件处理

上述设计简单易行，可避免较长的降级周期。但是，还有一些特殊事件没有考虑在内。在本小节中，将主要讨论如何处理可能发生的异常事件（即正常流程之外的事件），以确保预恢复的机制具有鲁棒性。

如图 6.3所列，下面将讨论四个特殊事件。在后续的说明中，为了更好的解释流程，设置了集中设备的状态，如下所述：（1）正样本设备，即被预测为故障的设备。其中又分为真阳性设备，即实际中确实会发生故障；假阳性设备，即实际中并不会发生故障。（2）负样本设备，即被预测为健康的设备。其中又分为真阴性设

中国知网　https://www.cnki.net

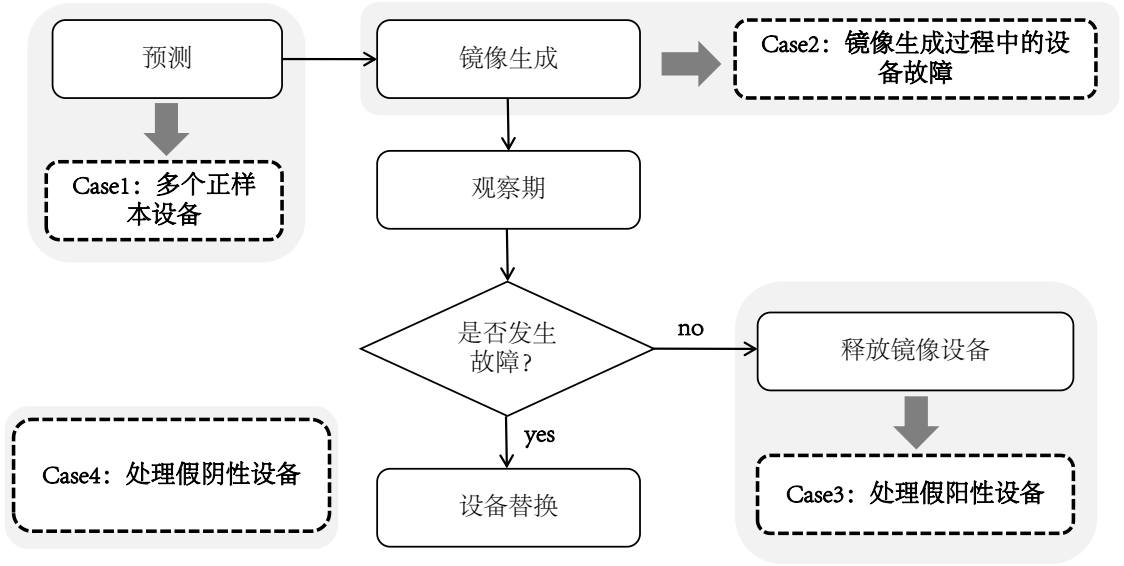


图 6.3 流程中可能存在的异常事件

备，即实际中不会发生故障；假阴性设备，即实际中会发生故障。第一个事件是多个正样本设备，当预测器给出的正样本设备多于服务器上可用的备用设备时，本文的预恢复机制就会出现无法为所有正样本设备提供镜像设备的情况。第二个事件是指在镜像生成过程中，某个设备出现故障，从而中断了镜像生成过程。第三个事件是可能的假阳性设备，这可能会毫无意义地占用镜像设备。第四个事件是可能的假阴性设备，对于没有预测出来的故障设备，预恢复机制并不能加以保护。下面将详细讨论这四种事件。

多个正样本设备

单个服务器上的多个存储设备有可能被预测为正样本设备。如上所述，每个正样本设备都需要一个备用设备来生成自己的镜像设备。因此，在预恢复的方案中，服务器上的备用设备数量必须不少于生成和维护镜像设备的正设备数量。然而，由于服务器上的存储设备插槽数量有限，可用的备用设备可能不够多。

一个乐观的解决方案是考虑到并非所有设备都需要紧急备份，因为在预恢复的机制中，预测精度会降低，出现假阳性的概率会提升。因此通过比较预测模型给出的设备概率值，决定需要紧急处理的设备。只有预测模型给出的最紧急设备才会生成镜像设备。其余设备将不做任何备份，并在发生故障时通过故障后恢复流

中国知网 https://www.cnki.net

程进行恢复。乐观解决方案的缺点是，未受保护的剩余设备仍有可能发生故障，导致 RAID 性能下降。

为了避免乐观解决方案依旧存在的 RAID 降级问题，预恢复机制采用了一种悲观的解决方案。悲观解决方案是将所有设备视为故障设备，直接替换无法生成镜像的正设备。预恢复机制首先使用可用的备用器件为部分正样本设备生成镜像。图 6.4展示了悲观解决方案一个例子。起初，前四个设备被预测为正设备，而服务器上只有两个备用设备可用。然后，预恢复机制会为设备 3 和设备 4 生成镜像设备，假定它们在预测模型中的概率值高于设备 1 和设备 2。

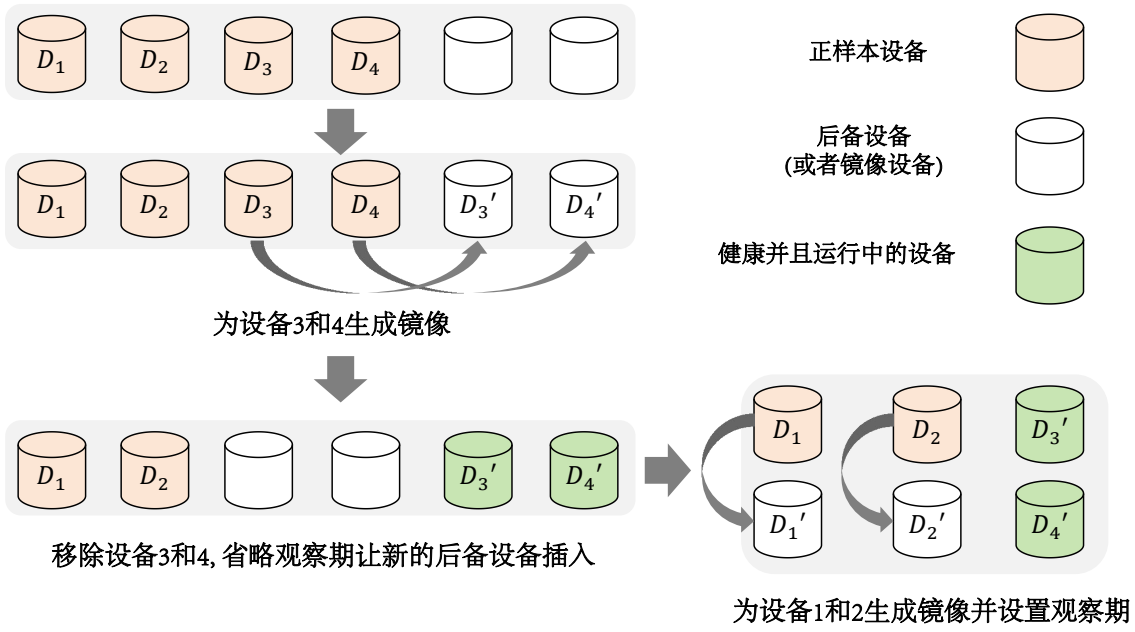


图 6.4　多正样本设备解决情况

由于采用的是悲观的解决方案，设备 3 和 4 被认为是有故障的。因此，它们会立即被相应的镜像设备取代。它们的观测周期被省略，因为它们被认为会在不久的将来失效，以便为其他设备的镜像设备腾出空间。更换了设备 3 和 4 后，它们的插槽就可以用来插入新的备用设备。观察期将被省略，直到有足够的备用设备为每一个剩余的正设备生成一个镜像设备。如果备用设备的数量足够多，则剩余的正设备在生成镜像设备后不会立即被移除，而是进入观察期。在给定的示例中，在替换了设备 3 和 4 后，备用设备的数量足够，因此设备 1 和 2 各有一个备用设备，用于生成镜像设备并进入观察期。需要注意的是，如果在正设备数量超过备用

中国知网　https://www.cnki.net

设备数量之前存在镜像设备，现有镜像设备的观察期将首先被省略。

与乐观的解决方案相比，这种悲观的解决方案可能会导致健康设备（假阳性设备）被替换。不过，它降低了数据丢失和阵列性能下降的概率，这是值得肯定的。此外，检测到的阳性设备数量多于备用设备数量的情况很少见。因此，这种悲观的解决方案不会造成太多健康设备的浪费。

镜像生成过程中的设备故障

在镜像生成过程完成之前，设备有可能发生故障。可以用一种直观的方式来处理这一事件。如果原始设备在镜像生成过程完成前发生故障，镜像设备上的数据可以取消，镜像设备可以直接用作 RAID 恢复的设备。在这种情况下，RAID 会进入降级模式，并执行故障后恢复。在启用预测的设计中，这种情况很少发生。因此，使用这种天真的策略是可以接受的。

处理假阳性设备

在预恢复机制中，为了获得高召回率，预测精度不可避免的被降低。这可能会导致预测过程将更多健康的存储设备识别为故障设备。预恢复机制还会为这些预测错误的设备生成镜像设备，然而它无法区分这些假阳性设备和真阳性设备。在这种情况下，镜像设备会被浪费掉，因为原始设备实际上并不需要备份。当其他存储设备随后被识别为故障时，这可能会导致服务器上的备用设备不足。备用设备的无意义损耗还会降低备用设备的耐用性，而没有任何好处。前面提到的观察期就是为了减少上述问题的影响而设置的。

关于观察期的设计，有两个问题需要讨论。首先，要确定观察期的长度。在本文中，观察期应与预测周期一致。预测周期指的是预测算法在训练模型时的目标预期，即多久之内会发生故障，这在先前的章节五中详细说明。因为观察期就是为了试探正样本设备在这段时间内是否会实际发生故障，因此将其设置成和预测周期一致是一个逻辑合理的选择。选择 10 天作为观察期的长度，是现有工作中使用的经验值，同时观察期并不限定，可以随时进行调整。

其次，应考虑对在观察期内存活的设备进行后续处理。尽管设备在观察期内

中国知网 https://www.cnki.net

幸存下来，但在随后的预测中仍可能被识别为阳性设备。这有两个原因。首先，预测模型依赖于设备的当前和历史统计信息，这些信息可能与前一次预测中的信息相似，从而导致设备再次被识别为阳性设备。其次，设备本身实际上可能是一个潜在的故障设备，其存活时间可能长于观察期。然而，能在观察期内存活下来的真正阳性设备并不多见。因此，我们的机制将所有在观察期内存活的设备都视为健康设备。这些设备会被标记，并禁止在这些设备上生成镜像设备。虽然这可能会使设备在真正失效时得不到保护，但由于这种情况很少见，因此影响很小，可以忽略不计。

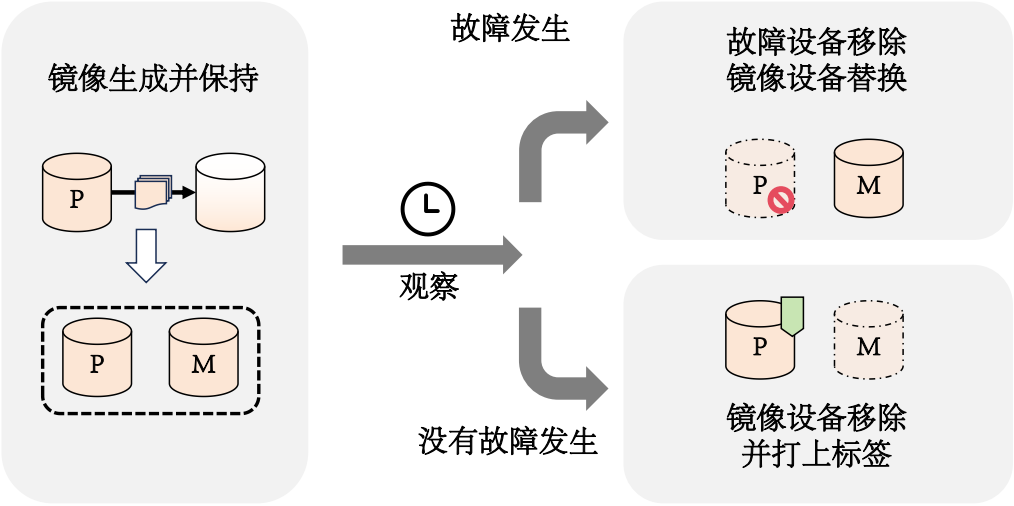


图 6.5　观察期和后续过程

观察期及其后续过程的全过程如图6.5所示。首先，为正样本设备生成镜像设备并备份。然后，两个设备进入观察期（10 天或其他时间），等待正样本设备出现故障。如果正样本设备在观察期内出现故障，则故障设备将被移除并由镜像设备取代。否则，正设备继续运行，并在观察期内存活。在这种情况下，当前镜像设备被释放，正样本设备被标记为健康。之后的预测结果可能会再次将其识别为阳性设备，但该标记将阻止预恢复机制为其生成新的镜像设备。需要强调的是，标记的作用是显著的，因为本身预恢复机制使用的是高召回率模型，对于高召回率模型来说，能够识别出更多的故障设备，但是会带来更多的误判，即健康设备识别为故障设备。因此，对于在观察期中没有发生实际故障的正样本设备来说，其本身是一个假阳性设备的概率在高召回率模型的情况下就会变大，因此为其打上标签，让

中国知网　https://www.cnki.net

其不再进入下一次观察期，能够很好的避免镜像设备重复被同一个假阳性设备消耗的情况，能够有效节省镜像资源。

处理假阴性设备

虽然预恢复机制提高了预测的召回率，但未预测的故障（即假阴性）仍然不可避免。这些未预测到的故障设备会被健康设备替换，并通过故障后恢复程序进行恢复。它们对镜像生成过程和观察期没有影响。

6.3 实验与测试

受实验资源的限制，无法在数据中心证明预恢复机制。因此，本文分两步进行评估。首先，本文在第 6.3.1 节中评估了故障前恢复机制在单 RAID 系统中的优势。在这一节中，本文通过禁用 RAID 中的一个存储设备来模拟设备故障。在此之后，将评估降级时间和对 RAID 性能的影响。其次，本文利用第6.3.1节中的降级评估结果和第6.2.1节中调整的预测模型的性能形成一个数值模型，以模拟第6.3.2节中数据中心的情况。最后使用数值模型来评估预恢复机制在数据中心中的总收益。

6.3.1 单 RAID 系统测试

实验环境及配置

如上所述，本文在一台配备两个 20 核 Intel XEON CPU 和 384GB DRAM 的服务器上进行了实验，该服务器运行 Ubuntu 22.04 和 Linux 内核 v6.2.0。本文使用四块 1TB ES3500P SSD 作为高密度 SSD，其中三块用于构建阵列，一块用于替换。为简单起见，只在每个固态硬盘上使用 128GB 分区来构建基于奇偶校验的 RAID。在 RAID 管理系统方面，本文采用了广泛使用的 Linux mdraid，并将 RAID5 作为四块固态硬盘的 RAID 格式。RAID 的块大小默认设置为 512KB，RAID 工作线程数设置为 16。对于前台工作负载，本文使用 FIO [48] 来生成读写工作负载。随机工作负载和顺序工作负载的 I/O 大小分别设置为 4KB 和 1MB。本文比较了建议的故障前恢复机制和传统的故障后恢复机制。对于预恢复机制，评估原型利用 mdraid 的

中国知网 https://www.cnki.net

RAID1 来生成和维护镜像设备。对于传统 RAID 恢复，故障设备由 mdraid 使用奇偶校验和剩余数据恢复。

降级性能对比

本节将比较预恢复机制和传统 RAID 恢复机制的降级周期长度和性能退化情况。如上所述，传统 RAID 恢复机制的降级周期指的是设备发生故障后 RAID 机制利用剩余数据通过奇偶校验原理进行数据恢复的时间段。对于本文提出的预恢复机制，当故障事先被预测到生成镜像设备并保留到故障发生时。在这种情况下，用镜像设备替换故障设备的时间不会超过一秒钟。然而，镜像生成过程持续时间较长，可能会对性能产生负面影响。因此，本文将预恢复机制的降级周期定义为镜像生成期。

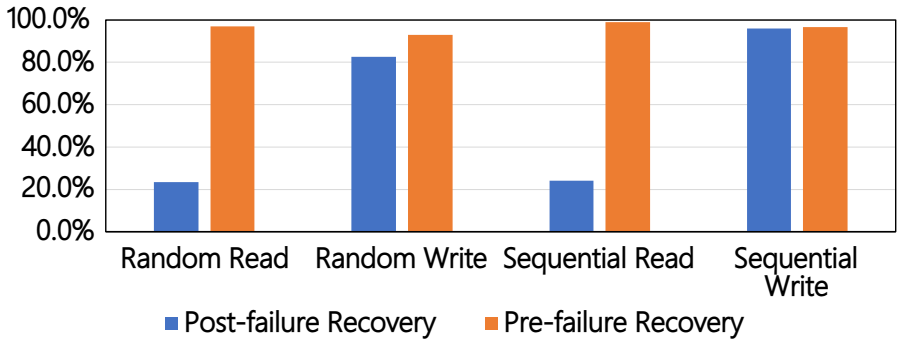


图 6.6　相对于正常吞吐量的降级吞吐量（越高越好）

降级吞吐量评估：本文首先测量了降级过程中的前台吞吐量（fio 线程数设置为 32，队列深度设置为 16，这使得我们实现的 RAID 系统的吞吐量达到饱和），以比较两种机制的性能。结果如图 6.6所示。对于故障前恢复机制而言，性能下降是微妙的。前台工作负载的吞吐量下降不低于正常吞吐量的 93%。镜像生成过程对前台工作负载的影响不大。相比之下，故障后恢复机制的吞吐量下降非常明显。随机读吞吐量下降到正常吞吐量的 23.4%。顺序读取吞吐量下降到正常吞吐量的 23.9%。这是因为故障设备上的数据丢失了，必须通过基于奇偶校验的恢复过程来恢复，该过程涉及读取所有剩余设备并进行计算。这一过程会大大降低吞吐量。

本文进一步研究了 4KB 随机读取访问的情况。通过修改队列深度和线程数工

中国知网　https://www.cnki.net

作参数，生成不同压力的工作负载。然后，分别在两种恢复方法的降级状态和正常状态下运行相同参数的工作负载，以进行评估。同一工作负载的吞吐量降级情况如图6.7所示。故障前恢复的性能衰减在正常性能的 96.9% 到 99.0% 之间，表现出微妙的衰减。相比之下，在低前台工作负载压力下，故障后恢复故障的性能可降至 72.0% 至 79.8%。随着前台工作负载压力的增大，故障后恢复故障下的 RAID 性能仅为正常性能的 23.4%。

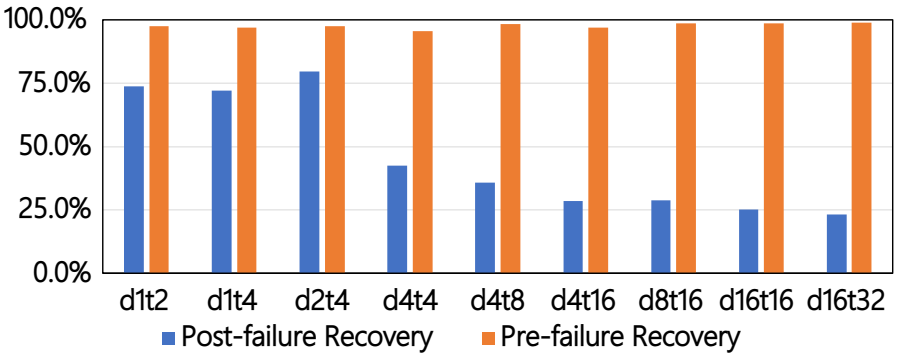
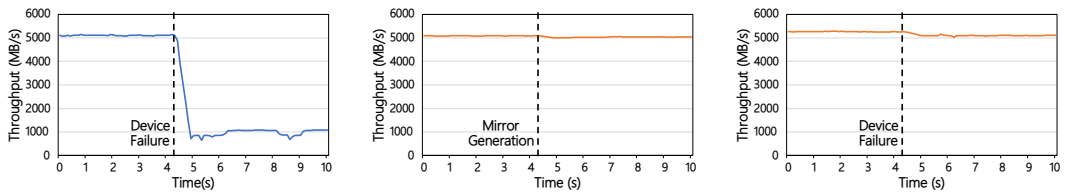


图 6.7 4KB 随机读取吞吐量的降级。对于 x 轴上的标签，*d* 指队列深度，*t* 指线程数，后面的数字分别是它们的值

本文还运行了一个 4KB 随机读取的 FIO 工作负载，并将设备设置为故障设备或开始镜像生成过程，以评估瞬态对性能的影响。图6.8显示了结果。在不涉及镜像生成的传统故障后恢复机制下，设备故障会直接导致性能下降，吞吐量锐减至正常吞吐量的五分之一左右。在所提出的故障前恢复机制下，首先对镜像生成过程进行评估。镜像生成过程只会导致低于 100MB/s 的性能下降，如图所示。镜像生成完成后，我们将原始设备设置为故障设备。如图6.8(c)所示，在预恢复过程中，



(a) 传统 RAID 恢复情况下的设备故障　(b) 镜像开始生成　(c) 预 RAID 恢复情况下的设备故障

图 6.8 设备故障和镜像生成对 4KB 随机读取吞吐量的瞬时影响

中国知网 https://www.cnki.net

设备故障仅造成 150MB/s 的吞吐量下降。这与故障后恢复机制相比是微妙的。需要注意的是，在镜像生成过程结束后，由于使用 RAID1 作为镜像维护方法，吞吐量会略有增加。因此，在已生成并维护镜像的图6.8(c)左半部分，性能比图6.8(b)高出约 150 MB/s。

表 6.2　降级时间情况，前台负载为 4kb 随机读写负载

	预恢复机制	**传统 RAID 恢复机制**
无前台工作负载（即空闲状态）	11min	11min
有前台工作负载	11min	41min

降级时间评估: 在这一部分，我们比较了两种机制的降解时间。结果如表6.2所示。我们首先测试了无前台工作负载（即空闲状态）时的降级时间。在这种情况下，两种机制的降级时间相似。降级时间都持续了约 11 分钟。由于两种情况下的带宽利用率都远低于最大值，因此理论上可以通过采用更积极的恢复策略来缩短降级持续时间。由于 RAID1 基于复制的恢复在理论上比基于奇偶校验的恢复更简单，因此在缩短降级持续时间的情况下，预恢复机制比传统 RAID 恢复更有利。但这里本文采用了默认的 mdadm 配置，这种配置比较保守，尽量不占用太多系统资源。然后，将 FIO 随机读取工作负载设置为前台工作负载。FIO 工作负载被设置为获取 RAID 系统的最大可用吞吐量（即 32 个线程，队列深度为 16），以保持 RAID 系统繁忙。在这种情况下，预恢复机制的衰减时间仍持续 11 分钟。但是，传统 RAID 恢复过程需要约 41 分钟，大大延长了时间。RAID1 的恢复过程（即镜像生成过程）比 RAID5 的奇偶校验计算简单，这保证了恢复速度。

总之，在预恢复过程中几乎不存在性能下降，而且性能下降的持续时间也大大缩短。需要注意的是，虽然当设备容量达到 TB 级时，预恢复仍会持续数十分钟或数小时，但预恢复的性能下降是微妙的。因此，预恢复机制的性能下降持续时间相对较短，几乎不会被察觉。

中国知网　https://www.cnki.net

6.3.2 集群规模评估

预测方案配置

为了验证预恢复设计在数据中心集群中的有效性，本文复现了一个故障预测模型来探究精确度和召回率的真实变化情况，以及在预恢复设计中能够带来的收益。本文使用随机森林算法（*ntrees*=100, *max_depth*=50）建立预测模型，该算法已被之前的工作采用，是目前解决固态硬盘故障预测最有效的算法。在上述参数中，*ntrees* 代表建立模型时要生成的树，*max_depth* 代表每棵树的最大深度。为了更好的复现性以及说明收益情况，数据使用了阿里巴巴公司公开的数据集 [9]，数据集中包含约 700000 块 SSD 以及超过 16000 块故障 SSD。正如第6.2.1节所述，在预测结果中能够得到样本被识别为故障设备的概率，通过调节故障概率，能够很轻松的实现模型在召回率和精确度上的调整。需要注意的是，预测模型并不是这项工作的重点，因此并不为模型进行除了先前提到的必要步骤外的优化。同时本文得到的最终效果和大部分先前工作类似。结果如图6.9所示。首先，可以清楚地看到这两个指标的" 跷跷板效应"。很难同时达到一个较高的值。为了展示预恢复设计效果，本文在结果中选择了三个典型点，即图中的 A、B 和 C。图6.9中列出了这三个点所对应的指标。

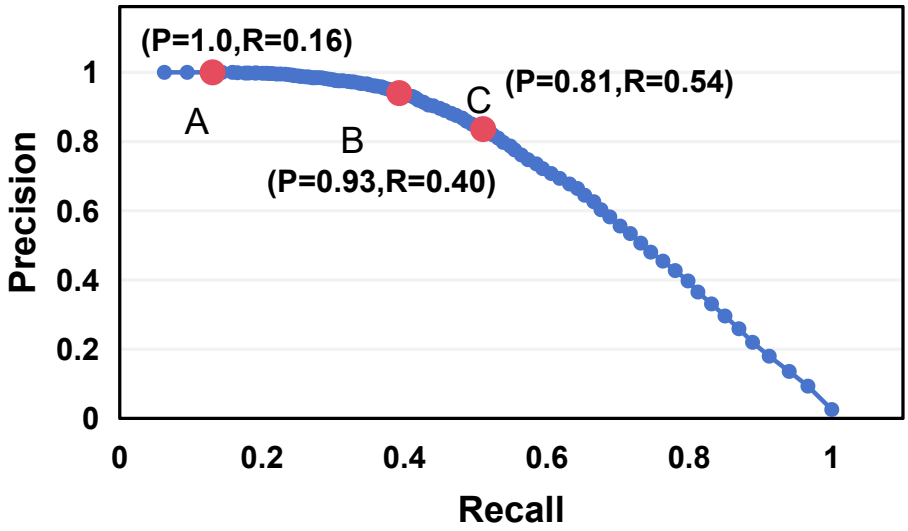


图 6.9 不同模型状态下的精确度和召回率趋势。标出了三个标志性的点，旁边列出的是其精确度和召回率

中国知网 https://www.cnki.net

这三个点代表三种不同的情况。A 点代表传统的预测应用场景，即不希望出现健康设备被错误分类的情况。而 B 点则代表了更平衡的情况，即召回率和精确度都处于相对较高的水平。而 C 点则是预恢复设计中提到的一种更激进的高召回率情况。随后的实验将根据这三点进行分析，以揭示预恢复设计的优越性。

集群数学建模

为了评估预恢复机制的优势，本文使用之前的结果来计算退化时间的减少。请注意，对于成功的预恢复，性能下降是微妙的，如第 6.3.1 节所示。因此，在本节中，只考虑传统 RAID 恢复的降级时间作为收益。

给定预测的召回率 r，可计算出减少的降级时间为 $t_{reduced} = r \times n_f \times t_d$、其中，$n_f$ 是整个数据中心的故障设备总数；而 t_d 是后恢复过程的平均降级时间。根据上述实验结果，本文将 n_f 设为 16,000，t_d 设为 11 分钟或 41 分钟。16000 是数据集中故障设备的真实数量。11 分钟和 41 分钟分别代表无前台负载运行和有前台负载运行时的降级时间。对于第 6.3.2 节中提到的三点，实验比较了它们带来的收益和开销。本文中将收益定义为总降级时间的减少量，即通过预恢复手段能在多长时间内保持 RAID 系统不降级。这在结果中以柱形显示。另一方面，开销是无效备份设备的数量，即即使设备最终没有发生故障，但是预恢复机制仍对其进行了备份。需要特别强调的是，这些用于备份的设备并没有被丢弃，尽管这里提到的这些设备数量被视为开销。它们最终仍会进入集群，以满足下一次备份需求。所以实际上的开销比实验中提及的要少得多。

如图6.10所示，与 A 点相比，B 点获得了很大的改进，降级时间的增幅减少了 150% 以上。然而，无效备份设备的数量仅增加了 481 台，与总体的 70 万台设备和 1.6 万台故障设备相比，几乎可以忽略不计。更激进的 C 点使收入提高了 238%，无效备份设备增加了 2,026 台。

性能和开销的权衡评估

如图6.10中前面的结果所示，虽然 C 点似乎能带来更高的收益，但同时也带来了更高的开销。因此，本节将专门讨论如何选择更合适的模型性能点。为了进一步

中国知网　https://www.cnki.net

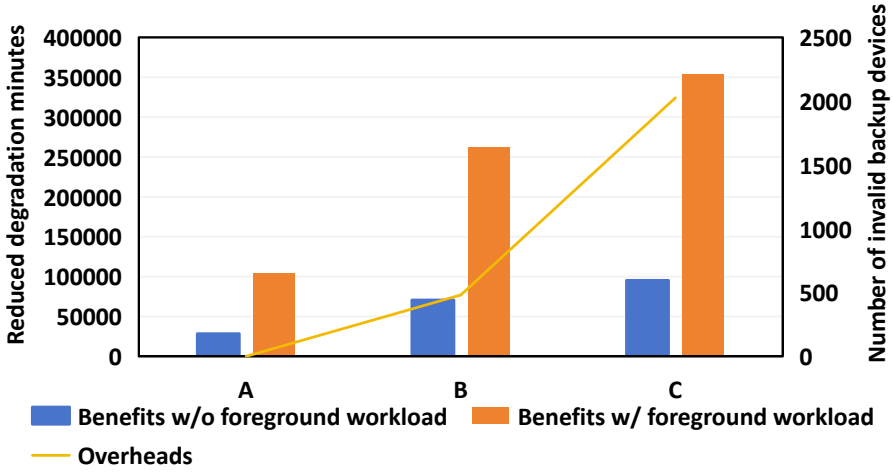


图 6.10 集群数学建模下的收益和开销对比

展示收益和开销的变化趋势，我们选取了 100 个模型性能点，这些点平均分布在图 6.9中。如图6.11所示，实验同时还计算了它们与 A 点相比所产生的开销和效益提升。

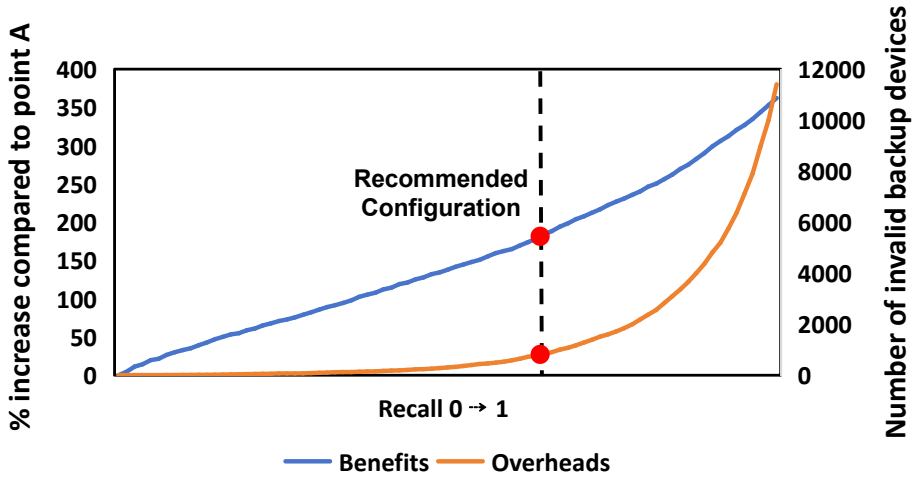


图 6.11 集群数学建模下的收益和开销对比趋势，图中标记了一个推荐配置点

可以看出，随着召回次数的增加，两者都呈上升趋势。不过，两者的变化速度有很大不同。曲线越往后，开销增长得越快，最终甚至达到了 12,000 个无效备份设备。图6.1可以说明，当调整阈值希望识别出所有故障设备时，不可避免地会将一些健康设备也判定为故障设备。而随着概率越来越低，收集到的健康样本也会越来越多，从而导致被误判的健康样本也越来越多。因此，本文建议将开销控制在故

中国知网 https://www.cnki.net

障设备的 5%，在我们的案例中为 800 个无效备份的设备编号，但能够获得 184% 的效益提升。

6.4　本章小结

本章提出了预恢复结合故障预测技术来代替传统 RAID 恢复，以减少 RAID 系统的降级时间。目前的预测方法强调精确性而忽视召回性，这限制了故障前恢复对总降级时间的减少。为解决这一问题，本文提出了一种两阶段备份机制，以提高预测的召回率，同时避免替换被误判为健康的存储设备。本章还讨论了异常情况，以证明所提方法的鲁棒性。最终的评估结果表明，两阶段备份机制可以显著减少数据中心的故障时间。通过预恢复的方式，降级性能仅仅只下降了约 150MB/s，相比较之下，传统 RAID 恢复机制则会有近 4000MB/s 的性能损失。同时，从集群角度来看，预恢复这种推崇搞召回率的方式，在仅增加了 481 台无效备份数量的情况下，可以多获得 150% 的收益提升。

中国知网　https://www.cnki.net

中国知网 https://www.cnki.net

第七章　总结与展望

SSD 这类存储设备的故障给数据中心带来的隐患很大，基于机器学习的故障预测手段能够通过预知即将发生故障的设备很好的避免数据丢失带来的性能问题以及服务宕机问题。目前，需要研究都着眼于数据选取、模型选择等研究角度，并没有将预测和 SSD 属性本身进行有效结合导致预测性能不佳，同时，也没有相关研究针对预测手段如何更好的部署进行优化。本文详细介绍了基于“状态”的 Diff-State 自动化数据预处理流程，在真实数据中心的数据验证下使得预测手段获得了很好的效果提升。同时，针对 RAID 这种常见数据中心应用场景提出了预恢复的机制将备份和故障预测手段相结合，以极小的代价，提升了故障预测手段的收益，极大程度减少了 RAID 自身数据恢复所带来的性能降级时间。

7.1　工作总结

本文基于华为数据中心真实数据，主要完成了以下内容：

（1）论文研究了真实数据中心场景下的 SSD 故障信息，依据闪存类型、工作负载及其他 S.M.A.R.T. 属性的分类对比分析。提出了故障原因的差异可能反映在某些 S.M.A.R.T. 属性的数值分布差异上的新发现。

（2）在此之上，论文提出了“状态”概念，即一组序列，这个序列中的各个值对应的是各个属性的分布区间。利用这种区分的方式减少数据集的混乱程度，同时潜在的区分一些故障原因。

（3）为了合理使用“状态”区分，论文提出了一种自动化数据预处理流程 Diff-State，通过状态特征提取、属性值划分、”状态”合并以及最后的数据填充四个步骤改进了传统 SSD 故障预测流程，得到了不错的效果提升。

（4）本文针对现有工作中对于样本选择的问题进行了优化，提出了带有恢复步长的回溯周期，很好的解决了健康样本不准确以及故障样本不及时的两个问题。

（5）在数据中心常见的 RAID 应用架构下，本文提出了预恢复结合故障预测技术来代替传统 RAID 恢复，以减少 RAID 系统的恢复数据时的降级时间。目前的预

中国知网　https://www.cnki.net

测方法强调精确性而忽视召回性，这限制了预测手段对故障设备的识别能力。为解决这一问题，本文提出了一种两阶段备份机制，以提高预测的召回率，同时避免替换被误判为健康的存储设备。最终的评估结果表明，两阶段备份机制可以显著减少数据中心的故障时间。

7.2 未来展望

本文的数据集来自于华为数据中心真实场景，但是属性种类的选择依旧有限。本文的工作无法获取到更加复杂的工作负载信息，只能够通过一些基础的负载信息进行推测。同时由于一些涉密原因，本文工作无法获取到实际故障设备，仅能获取到相关数据，这给更进一步研究故障具体原因带来了阻碍。基于此，下一步的工作是从更多的角度，收集更多的数据，研究更为细致的故障原因，而不是 Diff-State 潜在性的去进行区分。能够掌握一些具体故障原因，对故障预测甚至 SSD 设备的生产都有着重要意义。

本文针对于数据中心中 RAID 场景的优化，受限于实验室设备数量，仅能通过一些数学模型进行性能评估。在更大场景下进行模拟实现，能够更好的探索方案收益以及方案在大场景下可能存在的问题。基于此，下一步工作是通过更大规模的模拟实验对方案进行验证并加以优化。

中国知网 https://www.cnki.net

参考文献

[1] PINHEIRO E, WEBER W-D, BARROSO L A. Failure trends in a large disk drive population[J], 2007.

[2] SCHROEDER B, GIBSON G A. Understanding disk failure rates: What does an MTTF of 1,000,000 hours mean to you?[J]. ACM Transactions on Storage (TOS), 2007, 3(3): 8–es.

[3] SCHROEDER B, LAGISETTY R, MERCHANT A. Flash reliability in production: The expected and the unexpected[C] // 14th USENIX Conference on File and Storage Technologies (FAST 16). 2016: 67–80.

[4] MEZA J, WU Q, KUMAR S, et al. A large-scale study of flash memory failures in the field[J]. ACM SIGMETRICS Performance Evaluation Review, 2015, 43(1): 177–190.

[5] NARAYANAN I, WANG D, JEON M, et al. SSD failures in datacenters: What? when? and why?[C] // Proceedings of the 9th ACM International on Systems and Storage Conference. 2016: 1–11.

[6] ALTER J, XUE J, DIMNAKU A, et al. SSD failures in the field: symptoms, causes, and prediction models[C] // Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2019: 1–14.

[7] XU E, ZHENG M, QIN F, et al. Lessons and actions: What we learned from 10k {ssd-related} storage system failures[C] // 2019 USENIX Annual Technical Conference (USENIX ATC 19). 2019: 961–976.

[8] MANEAS S, MAHDAVIANI K, EMAMI T, et al. A Study of {SSD} Reliability in Large Scale Enterprise Storage Deployments[C] // 18th USENIX Conference on File and Storage Technologies (FAST 20). 2020: 137–149.

中国知网 https://www.cnki.net

[9] HAN S, LEE P P, XU F, et al. An {In-Depth} Study of Correlated Failures in Production {SSD-Based} Data Centers[C] // 19th USENIX Conference on File and Storage Technologies (FAST 21). 2021 : 417 – 429.

[10] LI P, DANG W, LYU C, et al. Reliability Characterization and Failure Prediction of 3D TLC SSDs in Large-Scale Storage Systems[J]. IEEE Transactions on Device and Materials Reliability, 2021, 21(2) : 224 – 235.

[11] XU F, HAN S, LEE P P, et al. General feature selection for failure prediction in large-scale SSD deployment[C] // 2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). 2021 : 263 – 270.

[12] ZHANG Y, HAO W, NIU B, et al. Multi-view Feature-based {SSD} Failure Prediction: What, When, and Why[C] // 21st USENIX Conference on File and Storage Technologies (FAST 23). 2023 : 409 – 424.

[13] HAN S, WU J, XU E, et al. Robust data preprocessing for machine-learning-based disk failure prediction in cloud production environments[J]. arXiv preprint arXiv:1912.09722, 2019.

[14] HAN S, LEE P P, SHEN Z, et al. Toward adaptive disk failure prediction via stream mining[C] // 2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS). 2020 : 628 – 638.

[15] CHAKRABORTTII C, LITZ H. Improving the accuracy, adaptability, and interpretability of SSD failure prediction models[C] // Proceedings of the 11th ACM Symposium on Cloud Computing. 2020 : 120 – 133.

[16] CHUNG T-S, PARK D-J, PARK S, et al. A survey of flash translation layer[J]. Journal of Systems Architecture, 2009, 55(5-6) : 332 – 343.

中国知网 https://www.cnki.net

[17] CAI Y, YALCIN G, MUTLU O, et al. Flash correct-and-refresh: Retention-aware error management for increased flash memory lifetime[C] // 2012 IEEE 30th International Conference on Computer Design (ICCD). 2012 : 94 – 101.

[18] CAI Y, GHOSE S, HARATSCH E F, et al. Error characterization, mitigation, and recovery in flash-memory-based solid-state drives[J]. Proceedings of the IEEE, 2017, 105(9) : 1666 – 1704.

[19] CAI Y, LUO Y, HARATSCH E F, et al. Data retention in MLC NAND flash memory: Characterization, optimization, and recovery[C] // 2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA). 2015 : 551 – 563.

[20] CAI Y, YALCIN G, MUTLU O, et al. Neighbor-cell assisted error correction for MLC NAND flash memories[J]. ACM SIGMETRICS Performance Evaluation Review, 2014, 42(1) : 491 – 504.

[21] CAI Y, YALCIN G, MUTLU O, et al. ERROR ANALYSIS AND RETENTION-AWARE ERROR MANAGEMENT FOR NAND FLASH MEMORY.[J]. Intel Technology Journal, 2013, 17(1).

[22] CAI Y, LUO Y, GHOSE S, et al. Read disturb errors in MLC NAND flash memory: Characterization, mitigation, and recovery[C] // 2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks. 2015 : 438 – 449.

[23] SUH K-D, SUH B-H, LIM Y-H, et al. A 3.3 V 32 Mb NAND flash memory with incremental step pulse programming scheme[J]. IEEE Journal of Solid-State Circuits, 1995, 30(11) : 1149 – 1156.

[24] JUNG T-S, CHOI Y-J, SUH K-D, et al. A 3.3 V 128 Mb multi-level NAND flash memory for mass storage applications[C] // 1996 IEEE International Solid-State Circuits Conference. Digest of Technical Papers, ISSCC. 1996 : 32 – 33.

中国知网 https://www.cnki.net

[25] JOO S J, YANG H J, NOH K H, et al. Abnormal disturbance mechanism of sub-100 nm NAND flash memory[J]. Japanese journal of applied physics, 2006, 45(8R): 6210.

[26] KURATA H, OTSUGA K, KOTABE A, et al. The impact of random telegraph signals on the scaling of multilevel flash memories[C] // 2006 Symposium on VLSI Circuits, 2006. Digest of Technical Papers.. 2006: 112 – 113.

[27] LUO Y, GHOSE S, CAI Y, et al. Enabling accurate and practical online flash channel modeling for modern MLC NAND flash memory[J]. IEEE Journal on Selected Areas in Communications, 2016, 34(9): 2294 – 2311.

[28] LUO Y, CAI Y, GHOSE S, et al. WARM: Improving NAND flash memory lifetime with write-hotness aware retention management[C] // 2015 31st Symposium on Mass Storage Systems and Technologies (MSST). 2015: 1 – 14.

[29] LUO Y, GHOSE S, CAI Y, et al. HeatWatch: Improving 3D NAND flash memory device reliability by exploiting self-recovery and temperature awareness[C] // 2018 IEEE International Symposium on High Performance Computer Architecture (HPCA). 2018: 504 – 517.

[30] LUO Y, GHOSE S, CAI Y, et al. Improving 3D NAND flash memory lifetime by tolerating early retention loss and process variation[J]. Proceedings of the ACM on Measurement and Analysis of Computing Systems, 2018, 2(3): 1 – 48.

[31] TAKEUCHI K, SATOH S, TANAKA T, et al. Memory-A Negative Vth Cell Architecture for Highly Scalable, Excellently Noise-Immune, and Highly Reliable NAND Flash Memories[J]. IEEE Journal of Solid State Circuits-Institute of Electrical and Electronics Engineers, 1999, 34(5): 675 – 684.

[32] MIELKE N, BELGAL H P, FAZIO A, et al. Recovery effects in the distributed cycling of flash memories[C] // 2006 IEEE International Reliability Physics Symposium Proceedings. 2006: 29 – 35.

中国知网 https://www.cnki.net

[33] ELERATH J G. AFR: problems of definition, calculation and measurement in a commercial environment[C] // Annual Reliability and Maintainability Symposium. 2000 Proceedings. International Symposium on Product Quality and Integrity (Cat. No. 00CH37055). 2000 : 71 – 76.

[34] ELERATH J G. Specifying reliability in the disk drive industry: No more MTBF's[C] // Annual Reliability and Maintainability Symposium. 2000 Proceedings. International Symposium on Product Quality and Integrity (Cat. No. 00CH37055). 2000 : 194 – 199.

[35] YANG J, SUN F-B. A comprehensive review of hard-disk drive reliability[C] // Annual Reliability and Maintainability. Symposium. 1999 Proceedings (Cat. No. 99CH36283). 1999 : 403 – 409.

[36] MANEAS S, MAHDAVIANI K, EMAMI T, et al. Operational Characteristics of {SSDs} in Enterprise Storage Systems: A {Large-Scale} Field Study[C] // 20th USENIX Conference on File and Storage Technologies (FAST 22). 2022 : 165 – 180.

[37] MASSEY JR F J. The Kolmogorov-Smirnov test for goodness of fit[J]. Journal of the American statistical Association, 1951, 46(253) : 68 – 78.

[38] HAO W, NIU B, LUO Y, et al. Improving accuracy and adaptability of SSD failure prediction in hyper-scale data centers[J]. ACM SIGMETRICS Performance Evaluation Review, 2022, 49(4) : 99 – 104.

[39] LIAW A, WIENER M, OTHERS. Classification and regression by randomForest[J]. R news, 2002, 2(3) : 18 – 22.

[40] SPEARMAN C. The proof and measurement of association between two things[J]. The American journal of psychology, 1987, 100(3/4) : 441 – 471.

[41] FEARNHEAD P. Exact and efficient Bayesian inference for multiple changepoint problems[J]. Statistics and computing, 2006, 16(2) : 203 – 213.

中国知网 https://www.cnki.net

[42] HSU H, LACHENBRUCH P A. Paired t test[J]. Wiley StatsRef: statistics reference online, 2014.

[43] H2O. H2O[R/OL]. . https://docs.h2o.ai/h2o/latest-stable/h2o-docs/index.html.

[44] LIU F T, TING K M, ZHOU Z-H. Isolation forest[C] // eighth IEEE international conference on data mining. 2008 : 413 – 422.

[45] CHEN T, GUESTRIN C. Xgboost: A scalable tree boosting system[C] // Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining. 2016 : 785 – 794.

[46] NATEKIN A, KNOLL A. Gradient boosting machines, a tutorial[J]. Frontiers in neurorobotics, 2013, 7 : 21.

[47] LINUX. Linux raid.[R/OL]. . https://raid.wiki.kernel.org/index.php/Linux_Raid.

[48] AXBOE J. FIO: Flexible I/O Tester[R/OL]. . https://github.com/axboe/fio.

中国知网 https://www.cnki.net